Simulación de ecosistema

Zorros, conejos y rocas: tres diseños en memoria compartida, secuencial, Pthreads y OpenMP. Informe del proyecto.

Curso: 300CIP013 Programación Paralela, 2026-II

Integrantes

[NOMBRE INTEGRANTE 1]

[NOMBRE INTEGRANTE 2]

Repositorio: https://github.com/EstebanBautista/MP3

Diseño 1, secuencial:
https://github.com/EstebanBautista/MP3/tree/diseno-1-secuencial

Diseño 2, estático:
https://github.com/EstebanBautista/MP3/tree/diseno-2-estatico

Diseño 3, adaptativo:
https://github.com/EstebanBautista/MP3/tree/diseno-3-adaptativo

Video: [ENLACE VIDEO]

Índice

1 Resumen y guía de lectura 3
1.1 Resumen 3
1.2 Mapa de la rúbrica y del enunciado 3
1.3 Código y ramas 4
2 Descripción del problema 4
2.1 Reglas de la simulación 5
2.2 Decisiones donde el enunciado no decide 5
2.3 Entrada y salida 6
3 Base común de los tres diseños 6
3.1 Por qué el resultado no depende del orden 6
4 Implementación de cada diseño 7
4.1 Diseño 1: secuencial 7
4.2 Diseño 2: estático con Pthreads 8
4.3 Diseño 3: adaptativo con OpenMP 9
5 Ventajas y desventajas de cada diseño 11
6 Preguntas por diseño 12
6.1 ¿Puede aprovechar múltiples núcleos, y por qué? 12
6.2 ¿Qué variables se comparten, y dónde podría haber una carrera? 12
6.3 ¿Qué estructuras de datos se usan, y se pueden mejorar? 13
6.4 ¿Qué tan fácil es extender el programa, y qué tan independientes son las entidades? 14
6.5 ¿Qué pasa con miles de entidades, y con entradas más grandes? 14
7 Verificación de la corrección 14
7.1 Qué se comprobó 14
7.2 Condiciones de carrera: qué se ejecutó y qué no 15
8 Resultados de rendimiento 16
8.1 Cómo se midió 16
8.2 Resultados principales 16
8.3 El coste de las barreras 19
8.4 Reparto adaptativo frente a estático 20
8.5 Linux en un contenedor sobre WSL2 21
9 Comparación: rendimiento, complejidad de desarrollo y mantenimiento 22
10 Experimentos del ecosistema 23
10.1 Qué se esperaba y qué se encontró 23
10.2 Método 23
10.3 Oscilación rápida 23
10.4 Un solo sobreimpulso inicial 24
10.5 Oscilación lenta débil con `GEN_PROC_FOXES=10` 25
10.6 Efecto de los parámetros en la estabilidad 25
10.7 Efecto de las rocas 27
10.8 Qué detecta el análisis, y qué no 27
11 Interfaz visual 28
12 Cómo ejecutar el programa 30
13 Conclusión 31
14 Apéndice: tablas completas de tiempos 31
14.1 Windows 10 nativo 32
14.2 Linux en un contenedor sobre WSL2 33

1 Resumen y guía de lectura

1.1 Resumen

Este proyecto simula un ecosistema de conejos, zorros y rocas sobre una matriz, con reglas deterministas, y compara tres diseños de la misma simulación en C++17: uno secuencial (D1), uno con Pthreads y bandas fijas (D2) y uno con OpenMP y bandas que siguen la carga (D3). Una página web local muestra la simulación y permite cambiar sus parámetros.

Los tres diseños producen el mismo resultado que los 7 casos de la tarea, con 1, 2, 4, 8 y 16 hilos. Los resultados principales son estos, y varios no son los que se esperaban:

- **Más hilos no siempre es más rápido.** El paralelismo solo da una aceleración clara en el mundo de 1000x1000: ×4,14 con Pthreads y 8 hilos. En 20x20 todos los diseños paralelos son más lentos que el secuencial, y en 100x100 el mejor resultado es ×1,66.
- **Con un hilo, los motores paralelos cuestan más que el secuencial:** entre 16,1 % a 33,2 %. No se investigó la causa.
- **El reparto adaptativo reduce el desequilibrio, pero no da una ganancia clara de tiempo.** Además, los dos casos "unbal" muestran poco desequilibrio incluso con reparto estático.
- **La comparación entre OpenMP y Pthreads depende de la plataforma.** En Windows gana Pthreads; en un contenedor Linux sobre WSL2 se invierte en 200x200, y no se demostró la causa.
- **Las poblaciones oscilan, pero no con el ciclo lento esperado.** Hay una oscilación rápida con periodo `GEN_PROC+1` y, con `GEN_PROC_FOXES=10`, una oscilación lenta débil. No se vio ningún ciclo lento fuerte a la sensibilidad del análisis, ni se demostró que lo observado sea una dinámica depredador-presa.

1.2 Mapa de la rúbrica y del enunciado

El enunciado de este proyecto no incluye rúbrica. Se usa la de los micro-proyectos anteriores del curso, que es la misma en los dos, y sus preguntas.

Tabla 1. Dónde está cada criterio de la rúbrica.

Criterio	Sección	Página
Implementación de cada diseño (30)	D1: 4.1. D2: 4.2. D3: 4.3. Base común: 3. Comprobación: 7	7
Ventajas y desventajas de cada diseño (20)	5	11
Preguntas por diseño (20)	6, una subsección por pregunta	12
Comparación de rendimiento, complejidad y mantenimiento (20)	9, con los datos de 8	22
Justificación de decisiones, estructuras y funciones (10)	Subsecciones "Justificación" de 4.1, 4.2 y 4.3. Reglas: 2.2	5
Informe y video (20)	Este documento. El enlace al video está en la portada	1

Tabla 2. Dónde está cada requisito del enunciado y de las reglas de entrega del curso.

Requisito	Sección	Página
Reglas de conejos, zorros, rocas, elección de celda y conflictos	2.1	5
Entrada por stdin y salida por stdout en el formato pedido	2.3	6
Versión secuencial primero, luego paralela en memoria compartida	4.1, 4.2, 4.3	7
Trabajadores con porción fija y sincronización con vecinos	4.2	8
Solución adaptativa que balancea la carga	4.3	9
Grilla, estadísticas, parámetros y controles de tiempo	11	28
Patrones cíclicos y efecto de los parámetros en la estabilidad	10	23
Tiempo de CPU y tiempo total	8, 14	16
Reflexión: más hilos no siempre es más rápido	13	31
Cada diseño en una rama, repositorio público y enlaces	Portada y 1.3	4
Código modular, orientado a objetos, con arreglos y `char*`	3	6
Cómo ejecutar el programa	12	30

1.3 Código y ramas

Cada diseño tiene su rama en el repositorio: diseno-1-secuencial, diseno-2-estatico y diseno-3-adaptativo. Cada una conserva el código tal como quedó al terminar ese diseño. Después, las partes comunes se reorganizaron: por ejemplo, las pasadas por banda salieron de `engine/sim_pth.cpp` a `engine/bands.hpp` cuando apareció el diseño 3.

La rama `main` reúne los tres diseños, la interfaz, los experimentos y este informe. Lo que describe este documento, y lo que se midió, es esa versión final del motor. Entre la rama del diseño 3 y esa versión, el motor solo cambió en comentarios y en la validación de la opción `--stats-file`.

2 Descripción del problema

El mundo es una matriz de R filas y C columnas. Cada celda está vacía o tiene un conejo, un zorro o una roca. A partir de un estado inicial se calculan N_GEN generaciones con reglas fijas, sin azar: la misma entrada debe dar siempre la misma salida. La dificultad para paralelizar es que el enunciado pide que todos los animales de una especie se muevan "al mismo tiempo", y que el orden en que se procesan no afecte el resultado, aunque varios quieran ocupar la misma celda.

2.1 Reglas de la simulación

Tabla 3. Reglas del enunciado. `X` es la fila, `Y` la columna y `G` la generación que se calcula, empezando en 0.

Tema	Regla
Orden	En cada generación se mueven primero todos los conejos y después todos los zorros.
Celda elegida	Se numeran desde 0, en orden norte, este, sur y oeste, las P celdas posibles. Se elige `(G+X+Y) mod P`.
Conejos	Pasan a una celda vacía adyacente. Sin celda vacía, se quedan.
Zorros	Comen un conejo adyacente si lo hay. Si no, se mueven a una celda vacía. Si tampoco hay, se quedan.
Procreación	Con edad de procreación de al menos `GEN_PROC_RABBITS` o `GEN_PROC_FOXES`, un animal que se mueve deja una cría en su celda, y ambos quedan con edad 0.
Hambre	Un zorro muere si pasan `GEN_FOOD_FOXES` generaciones sin comer. Muere tras no encontrar conejo y antes de intentar moverse.
Rocas	No se mueven y nadie puede ocuparlas.
Conflicto	Sobrevive el animal con mayor edad de procreación. Entre zorros empatados, el menos hambriento.

2.2 Decisiones donde el enunciado no decide

El enunciado deja algunos puntos abiertos. Para cerrarlos se usaron los archivos `allgen` de los ejemplos, que muestran cada generación. Se compilaron variantes de las reglas y se compararon con ellos. Esas compilaciones de prueba no están en el repositorio; sus resultados están en `experiments/results/rule_decisions.csv`.

Tabla 4. Variantes de las reglas comparadas con los archivos de ejemplo.

Variante	allgen5x5	allgen10x10	allgen20x20	allgen100x100	4 salidas mayores
Elegida: edad tras el reinicio y sin tope	idéntico	idéntico	idéntico	idéntico	iguales
Conejos: comparar antes del reinicio	idéntico	idéntico	difiere en gen. 870, celda (4,12)	difiere en gen. 1453, celda (56,9)	las 4 difieren
Zorros: comparar antes del reinicio	idéntico	idéntico	idéntico	idéntico	iguales
Zorros: comparar hambre antes de comer	idéntico	idéntico	idéntico	idéntico	iguales
Edad con tope en GEN_PROC	idéntico	idéntico	difiere en gen. 836, celda (0,17)	difiere en gen. 44, celda (91,32)	iguales

- **Conflicto entre conejos, decidido por los datos.** Se compara la edad después del reinicio por procreación, de modo que un conejo que acaba de procrear llega con edad 0 y pierde. Comparar antes del reinicio falla en `allgen20x20` y `allgen100x100`. Los dos órdenes discrepan en 3 de 9314 conflictos de conejos en 20x20, y en 5135 de 5959739 en 100x100.
- **Conflicto entre zorros, decisión de diseño.** Ni el enunciado ni los 7 casos deciden este punto. En ningún conflicto de zorros de los 7 casos cambia el ganador según se compare antes o después del reinicio (en 200x200 hay 87146 conflictos y 0 discrepancias). Se usó la misma regla que en los conejos, por coherencia. Un caso no incluido en los ejemplos podría distinguir las dos opciones, y cambiarla es una línea de código.
- **Hambre antes o después de comer: no cambia el resultado.** Los zorros que llegan a una misma celda vieron esa misma celda, así que o comieron todos o no comió ninguno. Si comieron todos, los que empatan en edad quedan idénticos. Si ninguno comió, sumar 1 a cada uno conserva el orden.

- **Las edades no tienen tope.** Un animal encerrado sigue envejeciendo por encima de `GEN_PROC`. Lo muestran los archivos: poner un tope falla en `allgen20x20` y `allgen100x100`, aunque no cambia las salidas finales, porque el animal se reinicia al moverse y nunca llega a un conflicto con esa edad.
- **Un animal por encima del límite procrea en su primer movimiento,** porque la condición es edad >= `GEN_PROC`. Una cría tiene ambas edades en 0, también si su padre no comió.

2.3 Entrada y salida

El programa lee de stdin una línea con `GEN_PROC_RABBITS`, `GEN_PROC_FOXES`, `GEN_FOOD_FOXES`, `N_GEN`, `R`, `C` y `N`, seguida de N líneas `OBJETO X Y`, y escribe el estado final en stdout con el mismo formato. Las medidas de tiempo van a stderr, de modo que no alteran la salida. La lectura y la escritura quedan fuera del tiempo medido.

3 Base común de los tres diseños

Los tres diseños comparten el modelo, las reglas y el programa principal. Solo cambia la clase que recorre el mundo. Por eso una diferencia de resultados solo podría venir del reparto o de la sincronización, y una diferencia de tiempo no viene de reglas distintas.

Tabla 5. Organización del motor.

Archivo	Contenido
`engine/world.hpp`	`Cell`, `Config` y `World`: las dos grillas en arreglos planos
`engine/rules.hpp`	Reglas de un animal, independientes de los hilos: `chooseNeighbor()`, `decideRabbit()`, `decideFox()`, `mergeRabbit()`, `mergeFox()`
`engine/simulator.hpp`	Interfaces `Simulator` y `Observer`, y las tres clases de simulador
`engine/cli.cpp`	`engineMain()`: opciones, lectura, cronómetro, escritura y línea de tiempos
`engine/io.cpp`	`readWorld()` y `writeWorld()`, con `FILE*` y `char*`
`engine/sim_seq.cpp`	Diseño 1
`engine/bands.hpp`	Bandas, buzones y pasadas, usados por los diseños 2 y 3
`engine/sim_pth.cpp`	Diseño 2
`engine/sim_omp.cpp`	Diseño 3 y su control estático

Cada ejecutable tiene un `main` mínimo: crea su simulador y llama a `engineMain()`. El nuevo diseño hereda de `Simulator`, y las opciones, los formatos y la medición de tiempo siguen siendo los mismos. Siguiendo la recomendación del curso, el motor usa arreglos en lugar de `vector` y `char*` en lugar de `string`.

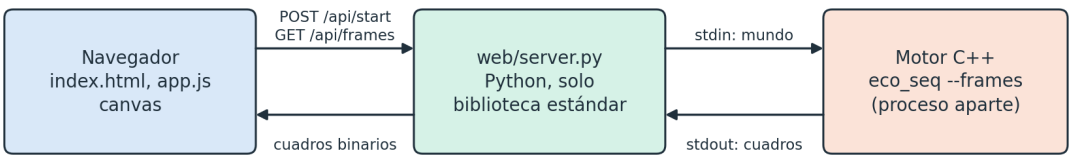


Figura 1. Componentes del programa. La interfaz es un proceso aparte del motor, y las mediciones de tiempo se hacen sin ella.

3.1 Por qué el resultado no depende del orden

Esta es la decisión central de todo el proyecto, y hace posibles los diseños paralelos:

1. **Una generación tiene dos subfases, conejos y zorros.** Cada una lee un estado que nadie escribe y construye uno nuevo (figura 2). Las celdas que se liberan durante una subfase no cuentan como vacías.
2. **La decisión no escribe en nada compartido.** `decideRabbit()` y `decideFox()` reciben el estado de lectura y devuelven un `Move`, con el destino, el animal con sus edades ya actualizadas y si deja una cría.
3. **La entrega es conmutativa.** `mergeRabbit()` y `mergeFox()` conservan al mejor candidato de una celda. Los animales de una especie tienen un orden total por sus edades, y dos candidatos empatados son idénticos, así que el resultado no depende del orden de llegada. Un zorro siempre reemplaza a un conejo, se entregue primero uno u otro.

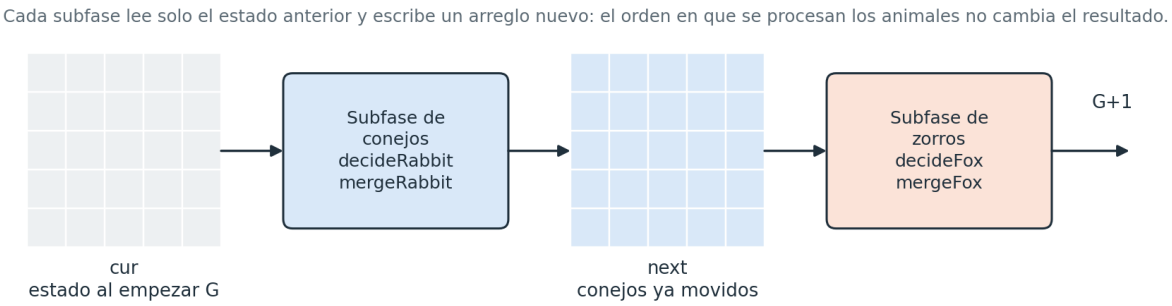


Figura 2. Las dos subfases de una generación sobre el doble búfer. El diseño 1 intercambia las grillas tras cada subfase; los diseños 2 y 3 alternan el sentido de lectura y escritura, sin intercambiar punteros compartidos.

La celda de una cría es la que dejó su padre. Ningún otro animal puede elegirla en esa subfase, porque el estado de lectura la muestra ocupada.

4 Implementación de cada diseño

4.1 Diseño 1: secuencial

Arquitectura. Un solo hilo, que es la referencia de todos los demás. `SeqSimulator::run()` llama, para cada generación, a `rabbitSubphase()` y `foxSubphase()`. Cada subfase limpia la grilla de destino, recorre todas las celdas e intercambia las grillas al terminar (figura 2).

Estructuras de datos. `World` tiene dos arreglos contiguos de `Cell`, `cur` y `next`, en orden de fila; `World::at` calcula el índice. `Cell` tiene el tipo en 1 byte y dos edades de 32 bits, 12 bytes en total con el relleno. Las dos grillas ocupan entonces 24 bytes por celda.

Funciones principales.

- `rabbitSubphase()` y `foxSubphase()`: recorrido del mundo.
- `decideRabbit()` y `decideFox()`: aplican las reglas a un animal.
- `applyRabbitMove()` y `applyFoxMove()`: entregan el resultado con la regla de conflicto.
- `carryRabbitSubphase()` y `carryFoxSubphase()`: copian lo que no actúa en la subfase. Un conejo que sobrevive a la subfase de zorros pasa por `mergeRabbit()`, de modo que nunca sobrescribe al zorro que lo comió.
- `clearGrid()`: limpia la grilla de destino.

Sincronización. Ninguna, porque hay un solo hilo.

Justificación. El enunciado pide empezar por una versión secuencial, y esta sirve de referencia exacta. Usa el doble búfer aunque no le haría falta para evitar carreras, porque es lo que hace que los animales se muevan "al mismo tiempo". Los arreglos planos dan acceso directo y recorrido

contiguo en memoria, y el tamaño del mundo solo se conoce al leer la entrada. Se recorre toda la grilla en orden de fila, sin una lista de animales, porque es lo más simple y el resultado no depende del orden.

4.2 Diseño 2: estático con Pthreads

Arquitectura. La grilla se divide en bandas horizontales contiguas, una por hilo, con tamaños que difieren como mucho en una fila (`staticBands()`). Los hilos son persistentes: se crean una vez en `PthSimulator::run()`, y el hilo que llama actúa como trabajador 0. Las bandas no cambian durante la corrida (figura 3).

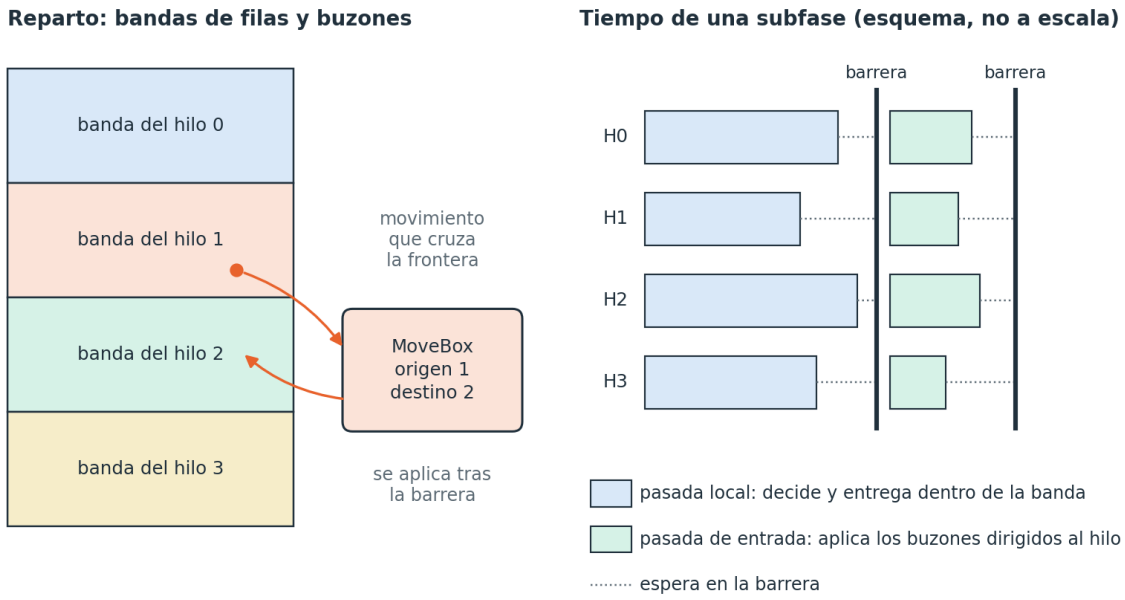


Figura 3. Diseño 2. Un movimiento que cruza la frontera de una banda no toca la grilla del vecino: va al buzón de ese origen y destino, y el dueño de la celda lo aplica después de la barrera. El tiempo es un esquema, no una medición.

Cada subfase tiene dos pasadas separadas por barreras:

1. **Pasada local** (`rabbitLocalPass()`, `foxLocalPass()`). Cada hilo limpia su banda en la grilla de destino y recorre su banda en la de origen. Un candidato cuyo destino está en su banda se aplica directamente; si no, va a un buzón.
2. **Barrera.** Todos los buzones están completos.
3. **Pasada de entrada** (`inboundPass<mergeRabbit>()`, `inboundPass<mergeFox>()`). Cada hilo aplica los buzones dirigidos a él y los vacía.
4. **Barrera.** La grilla de destino está completa y puede leerse.

Sin observador activo hay entonces 4 barreras por generación. Como hacer un intercambio de `w.cur` y `w.next` sería una escritura compartida, los hilos no lo hacen: la subfase de conejos va de `cur` a `next`, y la de zorros de `next` a `cur`.

Estructuras de datos.

- `Band`: las filas `first` a `last-1` de un hilo.
- `MoveBox`: arreglo de candidatos que crece al llenarse, con `alignas(64)`. Hay uno por cada par origen y destino, de modo que son hilos al cuadrado.
- `rowOwner`: arreglo con el hilo dueño de cada fila.
- `BandPlan`: agrupa lo anterior para que las pasadas no dependan de la biblioteca de hilos.
- `Shared` y `Worker`: el contexto de un hilo, con la barrera `pthread_barrier_t`.

Funciones principales. `PthSimulator::run()` reserva, crea y une los hilos. `workerMain()` es el bucle de cada hilo. `BandSink` decide entre aplicar y guardar en buzón, con `pushMove()`. `maxOverMean()` calcula el desequilibrio, como el mayor tiempo ocupado dividido por la media; las esperas en barrera no cuentan como tiempo ocupado.

Sincronización. Solo `pthread_barrier_wait`. No hay ningún mutex, porque cada escritura tiene un único dueño: ninguna celda la escriben dos hilos, y ningún buzón lo usan dos hilos a la vez. 6.2 detalla cada caso.

Justificación.

- **Bandas horizontales.** Es la división más simple de las que propone el enunciado. Un movimiento solo puede cruzar a la banda vecina, y en orden de fila cada banda queda contigua en memoria.
- **Hilos persistentes.** Se calculan miles de generaciones, y crear hilos en cada una añadiría ese coste todas las veces. No se midió esa alternativa.
- **Buzones por hilo y sin candados.** El enunciado sugiere sincronizar al final con los trabajadores adyacentes. Una alternativa era proteger las celdas de frontera con candados, pero eso añadiría una operación a cada acceso de frontera. Con buzones, la pasada local no se sincroniza nunca. La matriz completa de buzones usa más memoria de la necesaria en bandas fijas, pero permite usar el mismo código en el diseño 3, donde el vecino cambia.
- **Barrera y no variables de condición.** Las fases son globales y todos los hilos terminan cada una antes de empezar la siguiente, que es justo lo que expresa una barrera.

4.3 Diseño 3: adaptativo con OpenMP

Arquitectura. Usa las mismas pasadas y barreras que el diseño 2, de `engine/bands.hpp`. Una región `#pragma omp parallel` es el equipo de hilos persistentes, y cada pasada termina con una `#pragma omp barrier` explícita. No se usa `omp for`, porque las bandas y los buzones no son iteraciones de un bucle.

Lo nuevo es `rebalance()`. Cada `REBALANCE_EVERY` generaciones, empezando por la generación 0:

1. Cada hilo estima con `fillRowCosts()` el coste de sus filas: `C + ANIMAL_WEIGHT` × animales de la fila.
2. Barrera. Después, el hilo 0 calcula los cortes con `adaptiveBands()` y los dueños con `fillRowOwners()`.
3. Barrera. Todos ven las bandas nuevas.

`adaptiveBands()` recorre las filas una vez: cada banda toma filas mientras eso la acerque a una parte igual de lo que queda. Así, las bandas posteriores a una fila muy cara reparten el resto sin pagar por ella. En figura 4 se ve el efecto sobre un mundo real.

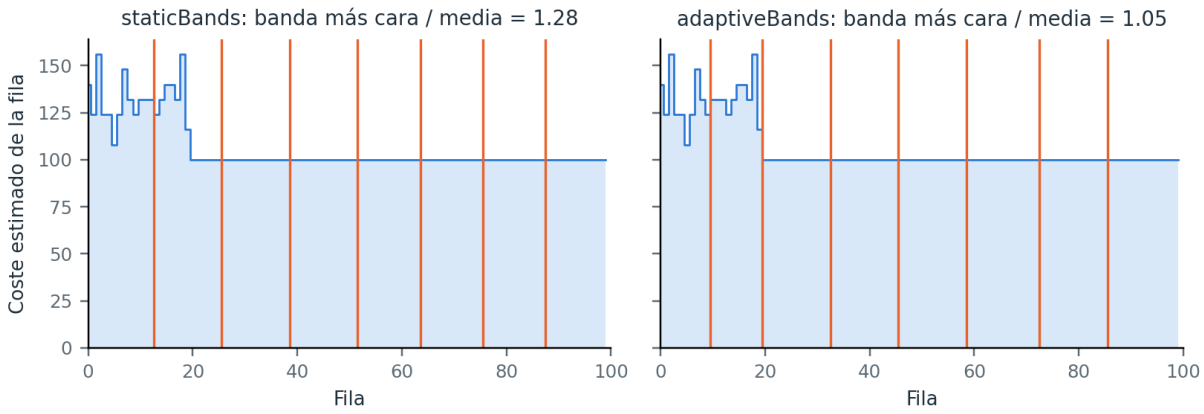


Figura 4. Cortes para 8 hilos en el mundo inicial de `input100x100_unbal01`, con la fórmula de coste del motor. Solo muestra la generación 0; en la corrida completa el desequilibrio medido con reparto estático es mucho menor que este (ver 8.4).

Estructuras de datos. Las mismas del diseño 2, más `rowCost`, un arreglo con el coste de cada fila. `Team` agrupa el contexto, y permite al hilo 0 cambiar `bands` y `rowOwner`, que las pasadas solo leen.

Funciones principales. `OmpSimulator::run()` prepara los datos y abre la región paralela. `simulate()` es el bucle de cada hilo. Además, `rebalance()`, `adaptiveBands()`, `fillRowCosts()` y `fillRowOwners()`. Las pasadas son las del diseño 2.

Sincronización. `#pragma omp barrier` tras cada pasada, dos barreras más en cada rebalanceo, y un `#pragma omp single` que hace el primer reparto cuando ya se conoce el equipo real. `omp_set_dynamic(0)` evita que el equipo cambie de tamaño, y si aun así es menor que el pedido (por ejemplo, con `OMP_THREAD_LIMIT`), las bandas se cortan para el equipo obtenido.

Justificación.

- **Rebalanceo entre generaciones.** En ese punto todos los buzones están vacíos y nadie usa las bandas anteriores. Cambiar las bandas solo cambia qué hilo calcula cada celda, y el nuevo dueño limpia la grilla de destino, así que el resultado no se altera.
- **Bandas contiguas, sin `schedule(dynamic)` por filas.** Con filas repartidas dinámicamente, una fila no tendría dueño conocido cuando otro hilo quisiera escribir en ella, y harían falta candados en las celdas. Conservar bandas mantiene el diseño sin candados.
- **OpenMP.** Una misma estructura se escribe con menos código de gestión de hilos, y usar los dos modelos permite comparar su coste de desarrollo y de ejecución.
- **Control estático.** Con `--schedule static` el diseño 3 usa `staticBands` y nunca rebalancea. No es un `schedule(static)` literal: como no hay `omp for`, esa cláusula no tendría dónde aplicarse, y reescribir el motor alrededor de un `omp for` cambiaría también la sincronización. Con este control, misma biblioteca, mismos buzones y mismas barreras, lo único que cambia es el reparto. Esto separa el efecto del reparto del efecto del modelo de hilos.

Elección de las constantes. `ANIMAL_WEIGHT` = 8 y `REBALANCE_EVERY` = 50 se eligieron midiendo los dos casos "unbal" con 8 hilos y 5 repeticiones (tabla 6). Para el peso, el desequilibrio mejora de 4 a 16 y empeora en 1 y 64, sin un óptimo marcado. Para el periodo, el tiempo cae rápido al rebalancear menos (cada rebalanceo añade dos barreras y un recorrido de la grilla), y 50 está entre los de menor desequilibrio. El barrido varía un parámetro cada vez: no es una búsqueda completa. Se hizo antes de congelar el motor y es independiente de las mediciones de 8.

Tabla 6. Barrido de las constantes del diseño 3 (`experiments/results/constants_sweep.csv`): mediana de 5 corridas con 8 hilos. El periodo está en generaciones.

Configuración	unbal01: tiempo (s) / desequilibrio	unbal02: tiempo (s) / desequilibrio
Estático, sin rebalanceo	1,732 / 1,062	1,700 / 1,042
Peso 1, cada 10	1,828 / 1,067	1,789 / 1,035
Peso 2, cada 10	1,836 / 1,045	1,846 / 1,020
Peso 4, cada 10	1,872 / 1,021	1,871 / 1,014
Peso 8, cada 10	1,880 / 1,012	1,853 / 1,014
Peso 16, cada 10	1,966 / 1,022	1,853 / 1,016
Peso 32, cada 10	1,882 / 1,020	1,853 / 1,026
Peso 64, cada 10	1,839 / 1,024	1,884 / 1,031
Peso 8, cada 1	2,722 / 1,033	2,769 / 1,042
Peso 8, cada 5	1,949 / 1,011	1,955 / 1,022
Peso 8, cada 25	1,824 / 1,007	1,789 / 1,020
Peso 8, cada 50	1,763 / 1,007	1,794 / 1,014
Peso 8, cada 100	1,744 / 1,017	1,768 / 1,014
Peso 8, cada 500	1,780 / 1,039	1,793 / 1,025
Peso 8, cada 1000	1,739 / 1,025	1,748 / 1,020

Incluso con los valores elegidos, el tiempo en estos dos casos no mejora frente al reparto estático. Se discute en 8.4.

5 Ventajas y desventajas de cada diseño

Tabla 7. Diseño 1: secuencial.

Ventajas	Desventajas
Es el más simple: 42 líneas propias y sin sincronización.	Usa un solo núcleo. En el mundo grande tarda 8,688 s, frente a 2,098 s con Pthreads y 8 hilos.
Es el más rápido en mundos pequeños: 0,004 s en 20x20, y a un solo hilo supera siempre a los otros dos diseños.	Su tiempo crece con el tamaño del mundo, sin forma de compensarlo.
No puede tener carreras, y sirve de referencia exacta para verificar los otros.	Recorre todas las celdas aunque el mundo esté casi vacío.
Consume el mínimo de CPU para el mismo trabajo.	

Tabla 8. Diseño 2: estático con Pthreads.

Ventajas	Desventajas
Fue el más rápido en Windows en casi todos los casos: en 200x200, 1,481 s frente a 2,110 s de OpenMP con 8 hilos.	Exige gestionar a mano los hilos, la barrera y la memoria: 104 líneas, más las 244 compartidas de `engine/bands.hpp`.
Sincronización explícita y fácil de seguir: 4 barreras por generación, sin candados.	Reparto fijo: si la carga se concentra en unas filas, no se adapta. En estos casos la pérdida medida es pequeña (desequilibrio de 1,067 en `unbal01` con 8 hilos).
Evidencia directa de carreras con ThreadSanitizer en su compilador: 0 avisos.	En mundos pequeños la barrera pesa más que el trabajo: en 20x20 con 16 hilos tarda 0,260 s, frente a 0,004 s del secuencial.
Pocas dependencias: solo una biblioteca POSIX de hilos.	En el contenedor Linux fue el peor en 200x200: 6,902 s con 16 hilos.

Tabla 9. Diseño 3: adaptativo con OpenMP.

Ventajas	Desventajas
Se adapta a la carga: en 1000x1000 con 8 hilos, baja el desequilibrio de 1,053 a 1,011.	Esa mejora no se traduce en una ganancia clara de tiempo, y en los casos "unbal" es algo más lento que el control estático.
Gestión de hilos más breve: el compilador crea el equipo, y el código de hilos es solo una región paralela y sus barreras.	Es el más complejo por lógica: añade costes, cortes, dueños cambiantes y dos barreras más cada 50 generaciones.
Fue el mejor en el contenedor Linux: ×6,74 en 200x200 con 16 hilos, aunque con mucha variación.	En Windows es el más lento con barreras: en 100x100 con 16 hilos, 3,703 s frente a 2,260 s de Pthreads.
Comparte las pasadas con el diseño 2, y un mismo binario hace de control con `--schedule static`.	Comprobar carreras es más difícil: la biblioteca de gcc no está instrumentada, y hace falta otra cadena de herramientas (7.2).

6 Preguntas por diseño

6.1 ¿Puede aprovechar múltiples núcleos, y por qué?

Diseño 1. No. Todo el trabajo lo hace un hilo, y el tiempo de CPU es prácticamente igual al tiempo total. En 1000x1000 son 8,672 s de CPU y 8,688 s de reloj.

Diseño 2. Sí, porque cada hilo calcula su banda de forma independiente y solo se coordina en las barreras. Pero aprovecharlos depende de que cada generación tenga trabajo suficiente frente al coste de las barreras. En 1000x1000 con 8 hilos acelera ×4,14, con eficiencia 0,52. En 100x100 el máximo es ×1,66 con 4 hilos, y con 16 hilos cae a ×0,46. Pasar de 8 a 16 hilos en este equipo significa usar núcleos lógicos (SMT), no físicos, y en 1000x1000 casi no mejora: 2,098 s a 2,032 s.

Diseño 3. Sí, por la misma razón, y en 1000x1000 alcanza una aceleración parecida (×3,94 con 8 hilos). En Windows, sin embargo, no llega a superar al secuencial en los mundos de 100x100: su máximo es ×0,96 con el control estático. En el contenedor Linux la situación cambia, como se ve en 8.5.

6.2 ¿Qué variables se comparten, y dónde podría haber una carrera?

Diseño 1. Ninguna, porque hay un solo hilo.

Diseños 2 y 3. Están en tabla 10, con el punto donde podría ocurrir una carrera y lo que la impide. 7.2 indica cómo se comprobó.

Tabla 10. Variables compartidas, posibles carreras y lo que las evita en los diseños 2 y 3.

Variable	Dónde podría ocurrir la carrera	Qué lo evita
Grilla de origen	Un hilo lee una celda de otra banda (un vecino) mientras alguien la escribe.	Nadie escribe la grilla de origen durante la subfase. La barrera final de la subfase anterior ordena esas escrituras antes de estas lecturas.
Grilla de destino	Dos hilos escriben la misma celda de una frontera.	Cada hilo solo escribe y limpia filas de su banda. Lo que cae en otra banda va a un buzón.
Buzones `MoveBox`	El origen añade mientras el destino lee, o varios orígenes escriben a la vez.	Hay un buzón por par origen y destino. El origen escribe en la pasada local, y el destino lee después de la barrera.
Contadores contiguos de los buzones	Compartición falsa (no es una carrera, pero cuesta tiempo).	`alignas(64)` separa cada buzón en su línea de caché.
`bands` y `rowOwner`	D3: el hilo 0 los cambia mientras otros los leen. En D2 no cambian tras crearse los hilos.	El cambio ocurre entre las dos barreras de `rebalance()`, cuando los demás esperan.
`rowCost` (D3)	El hilo 0 lee los costes mientras los demás los calculan.	Cada hilo escribe solo sus filas, y el hilo 0 lee tras la barrera.
`busy`	Un hilo publica su tiempo mientras se calcula el desequilibrio.	Cada hilo escribe su posición al terminar, y se lee tras unir los hilos o cerrar la región paralela.
Observador	Se leen los datos mientras otro hilo escribe.	Lo ejecuta un solo hilo, tras la última barrera y con los demás esperando en otra.
`team.plan.threads` (D3)	Se lee antes de conocerse el equipo.	Se escribe dentro de `omp single`, cuya barrera implícita precede a toda lectura posterior.

El error más sutil posible es un `rowOwner` desactualizado tras un rebalanceo: dos hilos escribirían una misma celda y la salida podría seguir siendo correcta. Por eso el resultado correcto no basta como prueba, y `tests/omp_bands_test.cpp` comprueba por separado que los dueños coinciden con las bandas.

6.3 ¿Qué estructuras de datos se usan, y se pueden mejorar?

Diseño 1. Dos arreglos planos de `Cell` y ninguna otra estructura durante la corrida. *Mejoras posibles, no probadas:* una lista de animales evitaría recorrer celdas vacías en mundos poco poblados, a cambio de mantenerla ordenada o de aceptar otro orden de entrega. Separar los tipos de las edades en dos arreglos reduciría la memoria en cada recorrido. Las edades no se pueden acortar sin cuidado, porque no tienen tope y un animal encerrado puede acumular tantas como generaciones.

Diseño 2. Además, `Band`, `rowOwner` y una matriz de `MoveBox`. *Mejoras posibles, no probadas:* como con bandas fijas un movimiento solo llega a una banda vecina, bastarían dos buzones por hilo y no hilos al cuadrado; se conservó la matriz para compartir el código con el diseño 3. Una división en dos dimensiones reduciría las fronteras en mundos muy grandes, pero añadiría vecinos y código.

Diseño 3. Además, `rowCost`. *Mejoras posibles, no probadas:* el coste es una estimación con un peso fijo, y se podría usar el tiempo real de cada banda. Rebalancear solo cuando el desequilibrio supere un umbral evitaría barreras inútiles. Dado que el desequilibrio medido ya es pequeño (8.4), estas mejoras tienen poco margen, y el mayor coste observado está en otra parte, las barreras.

6.4 ¿Qué tan fácil es extender el programa, y qué tan independientes son las entidades?

Las entidades no son objetos ni tienen hilo propio: son celdas de un arreglo, sin identidad. Su independencia viene de las reglas: dentro de una subfase, un animal decide viendo solo el estado de lectura y nunca sabe qué decidió otro. Solo interactúan en la entrega, cuando coinciden en una celda.

Diseño 1. Es el más fácil de extender. Cambiar una regla se hace en `engine/rules.hpp`. Una especie nueva necesita un valor en `CellKind`, sus campos en `Cell`, una función de decisión, una de entrega y una subfase más. Agregar más animales de las especies existentes no exige ningún cambio: es solo otra línea en la entrada.

Diseño 2. Cambiar una regla es igual de simple, porque también está en `engine/rules.hpp`. Una especie nueva necesita además una pasada local en `engine/bands.hpp` y 2 barreras más por generación en `workerMain()`. Hay que cuidar que su regla de entrega siga siendo conmutativa, porque de eso depende el resultado.

Diseño 3. Igual que el diseño 2, en `simulate()`. Si la especie nueva tuviera un coste distinto, también habría que revisar `fillRowCosts()`, que hoy da el mismo peso a conejos y zorros.

Agregar un diseño completo es relativamente sencillo en los tres casos: basta una clase que herede de `Simulator` y un `main` de pocas líneas.

6.5 ¿Qué pasa con miles de entidades, y con entradas más grandes?

En ningún diseño aumenta el número de hilos al aumentar las entidades, a diferencia de un diseño con un hilo por entidad. El coste de una generación es proporcional a las celdas más los animales, y la memoria a las celdas. Una grilla de 1000x1000 ocupa 24 MB en sus dos búferes. El mundo de 1000x1000 empieza con 25000 conejos, 5000 zorros y 5000 rocas, y se simuló en todos los diseños con la salida verificada. No se probaron mundos mayores, ni el comportamiento cerca del límite de la memoria.

Diseño 1. El tiempo por generación crece con el tamaño: 104 µs en 100x100, 400 µs en 200x200 y 8688 µs en 1000x1000. No hay forma de compensarlo.

Diseño 2. Más entidades favorecen al paralelismo, porque aumentan el trabajo entre barreras. La mejor aceleración sube con el tamaño: ×1,66 en 100x100, ×2,70 en 200x200 y ×4,27 en 1000x1000, y también sube el número de hilos que conviene usar. Con más movimientos que cruzan bandas, los buzones crecen, y se amplían duplicando su capacidad.

Diseño 3. Sigue la misma tendencia: ×0,95 en 100x100, ×1,93 en 200x200 y ×4,14 en 1000x1000. Cada rebalanceo recorre la grilla, de modo que su coste también crece con el mundo. Con pocas filas por hilo pueden quedar bandas vacías; las pruebas cubren esos mundos delgados.

7 Verificación de la corrección

7.1 Qué se comprobó

- **Salidas finales.** Los 3 diseños reproducen los 7 archivos `output*` de los ejemplos, con 1, 2, 4, 8 y 16 hilos en los diseños paralelos y con ambos repartos de OpenMP. La comparación es de bytes tras normalizar los finales de línea.
- **Todas las generaciones.** `eco_seq` reproduce `allgen5x5`, `allgen10x10`, `allgen20x20` y `allgen100x100`, las 10000 generaciones de este último, comparadas con `tools/diff_allgen.py`, que ignora finales de línea y líneas vacías al final. La comparación de `allgen100x100` se ejecuta aparte porque el archivo no se incluye en el repositorio. Los diseños paralelos se comparan generación por generación con el secuencial en los mundos pequeños.

- **Repetición.** 20 corridas seguidas de cada diseño paralelo con 8 hilos, para detectar resultados que cambien entre ejecuciones.
- **Casos límite.** Sin animales, sin generaciones, extinción, 16 hilos en mundos muy delgados, y más hilos que filas.
- **Reglas aisladas.** `tests/rules_test.cpp` prueba que la entrega da igual en cualquier orden, y `tests/omp_bands_test.cpp` prueba que los cortes siguen la carga en las generaciones correctas.
- **En cada medición.** Cada una de las mediciones de tiempo comparó su salida con la esperada, y ninguna falló. En el mundo grande, para el cual no hay salida oficial, la referencia es la salida de `eco_seq`.
- **En Linux.** La suite del motor también se ejecutó y pasó en un contenedor `gcc:14`, cuando se congeló el motor. Las pruebas añadidas después solo se ejecutaron en Windows.

La suite tiene 226 pruebas (`python -m unittest discover -s tests -t .`), y todas pasan en Windows con la versión final.

7.2 Condiciones de carrera: qué se ejecutó y qué no

ThreadSanitizer (TSan) no está disponible con MinGW, así que se usó en Linux, dentro de un contenedor Docker sobre WSL2. Los archivos con la evidencia están en `experiments/results`, y `scripts/tsan.sh` la reproduce.

Tabla 11. Corridas con ThreadSanitizer. Cada fila son 4 corridas: 20x20 (1000 generaciones) y `unbal01` (acortado a 200 generaciones), con 4 y 8 hilos.

Motor	Compilador y biblioteca	Avisos	Salida igual a la secuencial
`eco_pth`	gcc 14.4.0, TSan	0	sí
`eco_omp` con Archer	Ubuntu clang version 18.1.3 (1ubuntu1), libomp y Archer	0	sí
`eco_omp` sin Archer	gcc 14.4.0, libgomp sin instrumentar	408 (falsos positivos, ver abajo)	sí

Pthreads. En estas corridas no se observaron carreras. Se usó gcc 14.4.0, el compilador del contenedor de medición, no el g++ 16.2.0 de Windows.

OpenMP: la evidencia se apoya en Archer, no en la compilación con gcc. Con gcc, TSan da 408 avisos (113, 138, 55, 102 por corrida, en 32 ubicaciones distintas). libgomp, la biblioteca de OpenMP de gcc, no está instrumentada, de modo que TSan no ve el orden que imponen sus barreras. Se examinaron así:

1. Se agruparon los avisos por pares de accesos, y en cada par se comprobó que el código los separa con una `#pragma omp barrier`, con la barrera implícita de `omp single` o con el final de la región paralela. No apareció ningún par sin esa separación.
2. Se repitieron las corridas con Archer, la herramienta de LLVM que informa a TSan de las barreras de OpenMP, compilando con clang y libomp. Dieron 0 avisos.
3. Como control negativo, un programa pequeño en el mismo entorno da 0 avisos con barrera y 1 sin ella, así que Archer sí detecta una barrera ausente.
4. La cobertura de código confirma que esas corridas ejecutaron el rebalanceo: `adaptiveBands()` se llamó 48 veces, las esperadas.

Por eso se consideran falsos positivos, sin suprimir ninguno y sin cambiar el motor por ellos. Para OpenMP, por tanto, lo que se afirma es que en estas corridas con Archer no se observaron carreras. Clang y libomp no son la compilación con gcc y libgomp que se midió.

Lo que no se afirma ni se probó. TSan solo ve las carreras de las ejecuciones que observa, de modo que esto no demuestra que no existan. No se ejecutó con 16 hilos, con el control `--schedule static`, con los mundos 100x100 completos, 200x200 o 1000x1000, ni con `-O3` (se compiló con `-O1`). Tampoco se ejecutó en Windows. El argumento de fondo es el de tabla 10: cada escritura tiene un único dueño en cada fase.

8 Resultados de rendimiento

8.1 Cómo se midió

- **Equipo:** Intel(R) Core(TM) i7-10700F CPU @ 2.90GHz, 8 núcleos físicos y 16 lógicos, 32 GB de RAM, Windows 10 nativo.
- **Compilación:** g++ 16.2.0, `-O3 -std=gnu++17 -Wall -Wextra`, enlazado estático.
- **Mediciones:** 7 repeticiones por configuración, con orden alternado, en 6 mundos y con 1, 2, 4, 8 y 16 hilos. Se reporta la mediana y, entre corchetes, el rango intercuartil (IQR). La carga de CPU antes de empezar era de 0 %. Docker Desktop estaba abierto en segundo plano, sin contenedores en marcha.
- **Métricas:** tiempo total (reloj monótono), tiempo de CPU de todo el proceso, aceleración (`T_seq / T`, siempre respecto a `eco_seq`), eficiencia (aceleración / hilos) y desequilibrio (mayor tiempo ocupado / media). Sin interfaz y sin escritura de cuadros; la lectura y la escritura de archivos quedan fuera. En Windows, el tiempo de CPU tiene una resolución de unos 15,6 ms, de modo que en 20x20 no es fiable.
- **Mundo grande:** generado con semilla 20261031: 1000x1000, con las densidades de `input200x200`, 1000 generaciones.

Las tablas completas están en 14, y los datos crudos en `experiments/results`.

8.2 Resultados principales

Tabla 12. Mejor tiempo de cada modelo por mundo, en Windows, y su aceleración respecto a `eco_seq`.

Mundo	eco_seq (D1)	Mejor Pthreads (D2)	Mejor OpenMP (D3 o control)
20x20	0,004 s	0,005 s con 1 hilo (×0,77)	0,005 s con 1 hilo (×0,75)
100x100	1,043 s	0,630 s con 4 hilos (×1,66)	1,090 s con 4 hilos (×0,96), control estático
100x100 unbal01	0,746 s	0,559 s con 4 hilos (×1,33)	0,901 s con 1 hilo (×0,83), control estático
100x100 unbal02	1,019 s	0,652 s con 4 hilos (×1,56)	1,088 s con 4 hilos (×0,94), control estático
200x200	3,999 s	1,481 s con 8 hilos (×2,70)	2,061 s con 4 hilos (×1,94), control estático
1000x1000	8,688 s	2,032 s con 16 hilos (×4,27)	2,097 s con 16 hilos (×4,14)

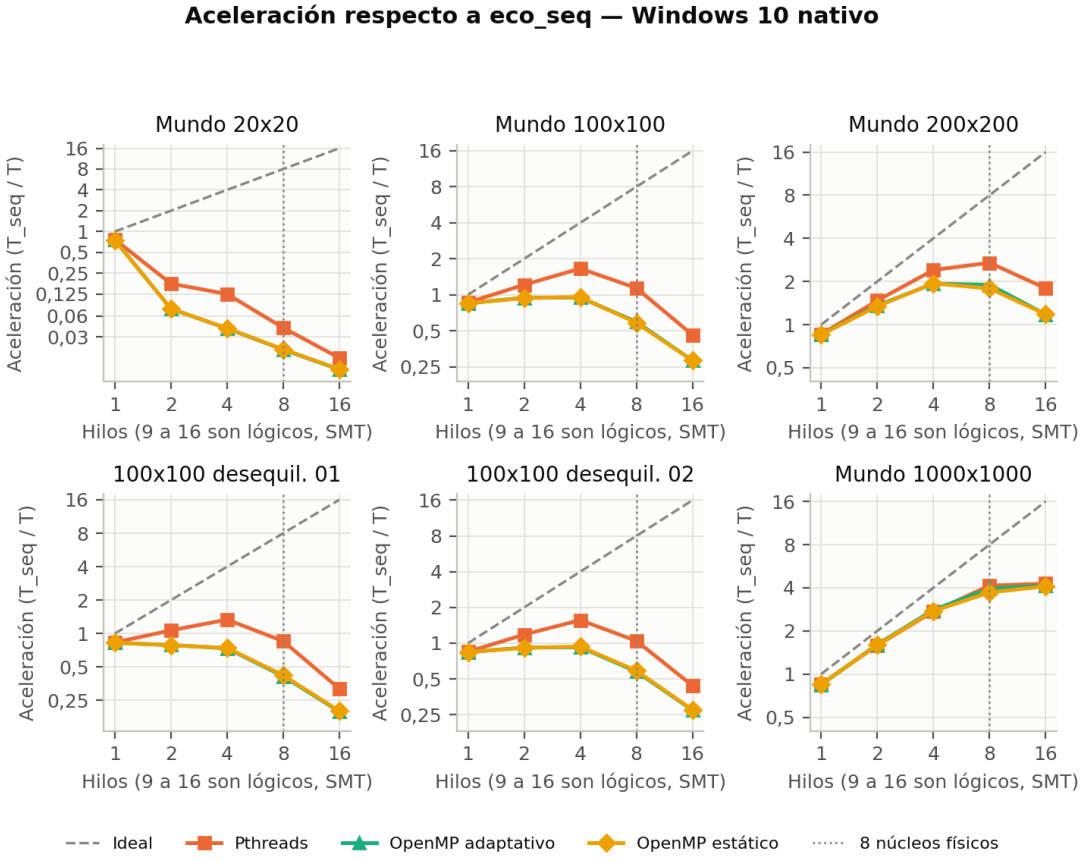


Figura 5. Aceleración respecto a `eco_seq` en Windows, en escala logarítmica. Por debajo de 1 es más lento que el secuencial. Las líneas de OpenMP adaptativo y estático casi coinciden.

El tamaño del mundo decide. En 20x20, todos los diseños con 2 o más hilos son más lentos que el secuencial: 0,260 s con Pthreads y 0,382 s con OpenMP a 16 hilos, frente a 0,004 s. En 100x100 solo Pthreads gana, con un máximo de ×1,66. En 200x200, Pthreads llega a ×2,70 con 8 hilos. El único caso con aceleración clara en todos los diseños es 1000x1000.

Con 16 hilos no se gana casi nunca. En Windows, 16 hilos son más lentos que 8 en todos los mundos, salvo en 1000x1000, donde hay un empate dentro del IQR. El equipo tiene 8 núcleos físicos, y del hilo 9 al 16 son núcleos lógicos.

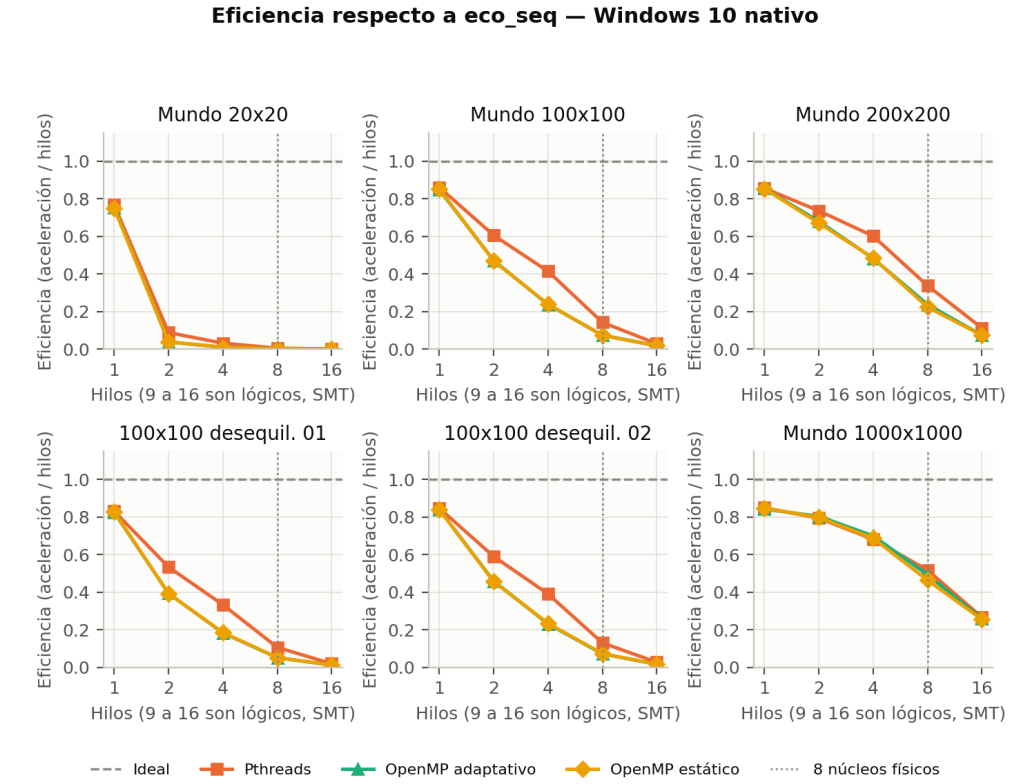


Figura 6. Eficiencia en Windows.

Más hilos consumen más CPU para el mismo trabajo. En 1000x1000 el tiempo de CPU sube de 10,234 s con 1 hilo a 14,875 s con 8 y 25,625 s con 16, en Pthreads. El tiempo de reloj baja, pero con un coste creciente.

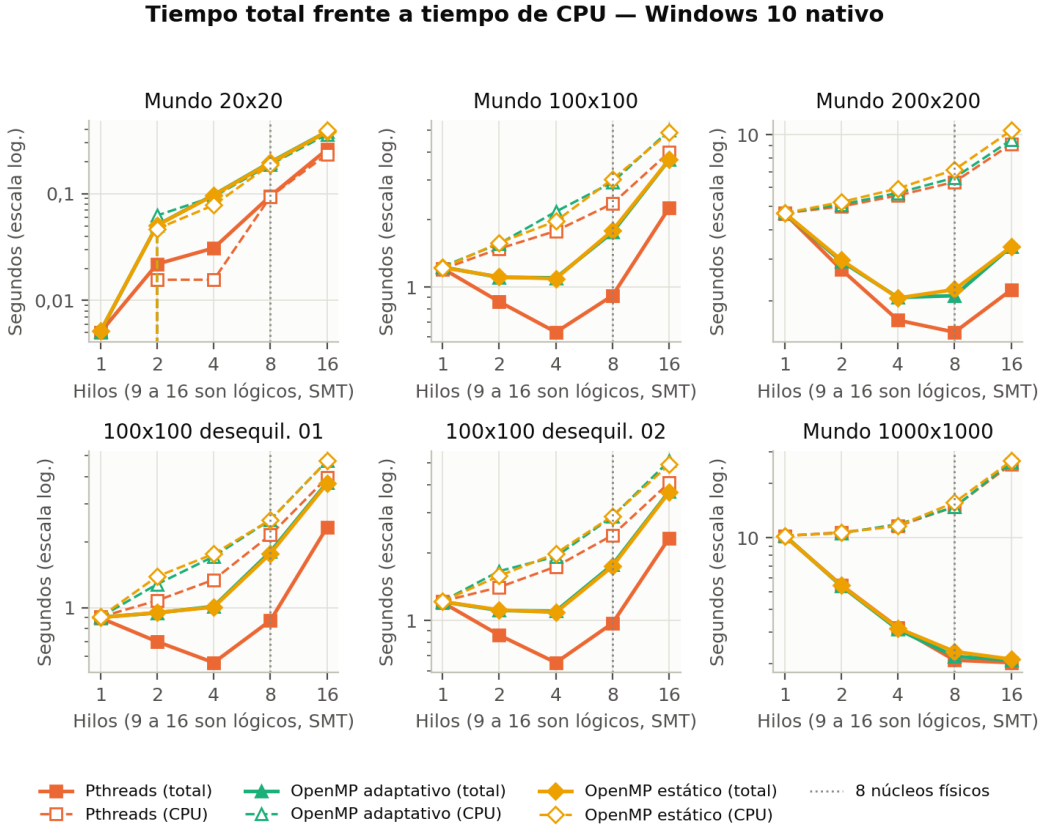


Figura 7. Tiempo total y tiempo de CPU en Windows.

Sobrecoste con un hilo. Con un hilo, sin ninguna posibilidad de paralelismo, `eco_pth` y `eco_omp` son más lentos que `eco_seq`: 16,1 % a 21,2 % desde 100x100 hasta 1000x1000, y 30,3 % a 33,2 % en 20x20. En 1000x1000: 8,688 s frente a 10,227 s. Es estable, no es ruido, y también aparece en el contenedor Linux (21,4 % a 25,2 %), pero no se investigó la causa y no se afirma ninguna.

8.3 El coste de las barreras

Tabla 13. Coste de una barrera vacía en µs (mediana de 7 repeticiones de 40 000 barreras, `experiments/barrier_bench.cpp`).

Hilos	Windows, Pthreads	Windows, OpenMP	Contenedor, Pthreads	Contenedor, OpenMP
2	4,3	12,8	16,2	0,2
4	6,9	24,1	43,6	0,2
8	25,3	47,9	83,4	0,3
16	66,6	92,9	159,3	0,4

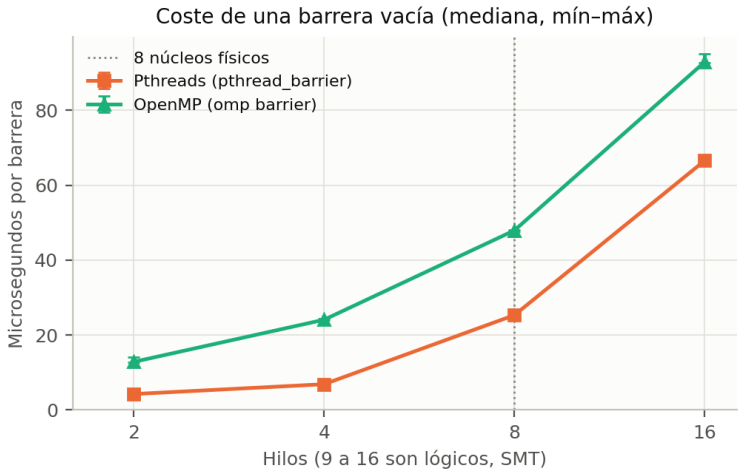


Figura 8. Coste de una barrera vacía según el número de hilos.

En Windows, una barrera de OpenMP cuesta más que una de Pthreads con cualquier número de hilos. Por el código, cada generación tiene 4 barreras (más 2 cada 50 generaciones en el modo adaptativo). A 16 hilos, 4 barreras vacías costarían unos 266 µs en Pthreads y 372 µs en OpenMP. tabla 14 las compara con el tiempo medido por generación.

Tabla 14. Tiempo por generación en Windows, comparable con el coste de 4 barreras vacías a 16 hilos.

Mundo	Pthreads, 16 hilos	OpenMP estático, 16 hilos	eco_seq
20x20	260 µs	379 µs	4 µs
100x100	226 µs	371 µs	104 µs
200x200	223 µs	338 µs	400 µs
1000x1000	2032 µs	2123 µs	8688 µs

En 20x20, 100x100 y 200x200, el tiempo por generación a 16 hilos es del mismo orden que 4 barreras, aunque el secuencial necesita mucho menos en los dos primeros. En 1000x1000 es varias veces mayor. Es coherente con que en mundos pequeños domine la sincronización y en los grandes el cálculo. Pero es una comparación de órdenes de magnitud y no una causa demostrada: el microbenchmark usa barreras vacías, y a veces da más que el tiempo medido en el programa real (266 µs frente a 226 µs por generación en Pthreads con 100x100).

8.4 Reparto adaptativo frente a estático

El diseño 3 se comparó con su control estático, de modo que el modelo de hilos sea el mismo. El resultado va en contra de lo esperado:

Tabla 15. Desequilibrio (mayor tiempo ocupado / media) y tiempo en OpenMP, Windows. Entre corchetes, el IQR.

Mundo	Hilos	Deseq. estático	Deseq. adaptativo	Tiempo estático (s)	Tiempo adaptativo (s)
100x100 unbal01	8	1,063	1,010	1,772 [0,012]	1,806 [0,034]
100x100 unbal01	16	1,087	1,020	3,715 [0,042]	3,755 [0,032]
100x100 unbal02	8	1,046	1,019	1,739 [0,028]	1,768 [0,036]
100x100 unbal02	16	1,093	1,028	3,705 [0,041]	3,736 [0,105]
200x200	8	1,058	1,007	2,239 [0,177]	2,110 [0,070]
1000x1000	8	1,053	1,011	2,340 [0,158]	2,203 [0,104]
1000x1000	16	1,038	1,017	2,123 [0,184]	2,097 [0,151]

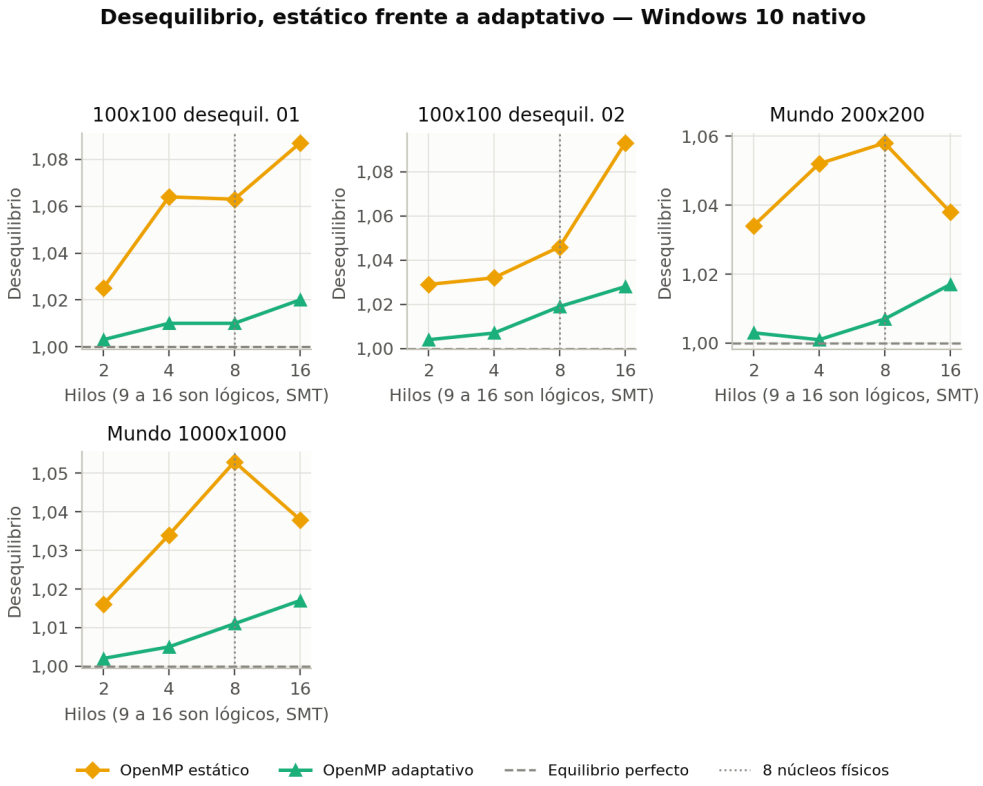


Figura 9. Desequilibrio medido en Windows.

- **El adaptativo reduce el desequilibrio.** Lo hace en todos los casos de la tabla, de modo que el mecanismo funciona.
- **Pero no da una ganancia clara de tiempo.** En 1000x1000 con 8 hilos, 2,203 s frente a 2,340 s, una diferencia del orden del IQR. En los dos casos "unbal" es ligeramente más lento.
- **Los casos "unbal" no están muy desequilibrados en la corrida completa.** Con reparto estático su desequilibrio es de 1,063 y 1,046 a 8 hilos, de modo que hay poco que recuperar, aunque en el mundo inicial la carga sea muy desigual (figura 4). No se midió cómo evoluciona la carga durante la corrida, y por qué se iguala queda sin probar.

No se aisló la causa de la falta de ganancia. Con estos datos, no hay ventaja de tiempo que justifique la complejidad adicional.

8.5 Linux en un contenedor sobre WSL2

También se midió en un contenedor `gcc:14` (GCC 14.4.0) sobre Docker Desktop y WSL2, en el mismo equipo y con 7 repeticiones, solo en 200x200 y 1000x1000. **No es Linux nativo**, y estas cifras no se mezclan con las de Windows. Las banderas de compilación son las mismas, salvo el enlazado estático. El contenedor no tenía límite de CPU.

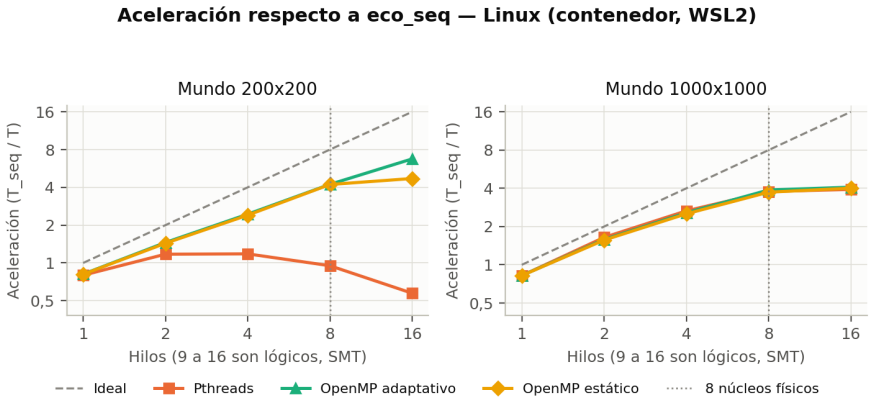


Figura 10. Aceleración en el contenedor Linux sobre WSL2.

El orden entre OpenMP y Pthreads se invierte en 200x200. Con 16 hilos, OpenMP adaptativo tarda 0,587 s [0,214] (×6,74), el estático 0,841 s [0,430] y Pthreads 6,902 s [0,220] (×0,57). En Windows, el mismo caso da 2,230 s en Pthreads y 3,393 s en OpenMP. En 1000x1000 el contenedor se parece a Windows: ×3,92 en Pthreads y ×4,09 en OpenMP adaptativo.

Salvedades. El valor de OpenMP a 16 hilos en 200x200 es muy variable: entre 0,554 s y 1,078 s. La mejora de 8 a 16 hilos en ese caso no aparece en 1000x1000, y no está explicada.

La causa no está demostrada. Las barreras en el contenedor también cambian mucho (tabla 13): Pthreads es más caro que en Windows, lo que va en la misma dirección. El valor de OpenMP, de 0,2 a 0,4 µs, es tan bajo que puede reflejar la forma del microbenchmark, con barreras vacías seguidas, más que el coste con trabajo real, y por eso no se usa como causa. Entre las dos plataformas difieren el compilador, la biblioteca de OpenMP, el sistema operativo y la virtualización, y sus efectos no se separaron.

9 Comparación: rendimiento, complejidad de desarrollo y mantenimiento

Tabla 16. Comparación de los tres diseños.

Aspecto	D1: secuencial	D2: estático con Pthreads	D3: adaptativo con OpenMP
Rendimiento en mundos pequeños (20x20)	El mejor: 0,004 s	0,022 s con 2 hilos	0,051 s con 2 hilos
Rendimiento en mundos medianos (200x200)	3,999 s	El mejor en Windows: 1,481 s con 8 hilos	2,071 s con 4 hilos en Windows; el mejor en el contenedor
Rendimiento en mundos grandes (1000x1000)	8,688 s	2,098 s con 8 hilos	2,203 s con 8 hilos
Complejidad: código propio	42 líneas	104 líneas más `engine/bands.hpp` (244)	128 líneas más `engine/bands.hpp`
Complejidad: sincronización	Ninguna	5 llamadas a barrera en el código; 4 por generación	7 directivas de barrera en el código; 4 por generación, más 2 cada 50
Complejidad: gestión de hilos	Ninguna	Manual: crear, unir, inicializar la barrera y reservar la memoria	La hace OpenMP: una región paralela, pero debe tratar un equipo menor que el pedido
Complejidad: razonar la corrección	Directa	Dueños fijos: basta revisar las fases	Dueños que cambian: hay que razonar además el punto de rebalanceo
Complejidad: verificar carreras	No aplica	TSan con gcc da 0 avisos	Requiere Archer con clang: gcc da 408 falsos positivos
Mantenimiento: cambiar una regla	Un archivo: `engine/rules.hpp`	El mismo archivo; hay que conservar la entrega conmutativa	Igual que D2
Mantenimiento: especie nueva	Una subfase	Una subfase, con 2 pasadas y 2 barreras más	Igual que D2, y revisar la estimación de coste
Mantenimiento: riesgos	Bajo	Errores de sincronización que no siempre cambian la salida	Lo mismo, y además un `rowOwner` desactualizado; la ganancia depende de constantes ajustadas para estos casos
Mantenimiento: portabilidad del rendimiento	Tiempos parecidos en las dos plataformas: 3,999 s y 3,958 s en 200x200	Peor en el contenedor Linux	Mejor en el contenedor Linux y peor en Windows

Tabla 17. Tamaño de los archivos que distinguen a cada diseño.

Archivo	Usado por	Líneas (con comentarios)
`engine/world.hpp`	Modelo común	35
`engine/rules.hpp`	Reglas comunes	184
`engine/sim_seq.cpp`	D1	42
`engine/bands.hpp`	D2 y D3	244
`engine/sim_pth.cpp`	D2	104
`engine/sim_omp.cpp`	D3	128

Compartir las reglas y las pasadas fue lo que más redujo el mantenimiento. Sin `engine/bands.hpp`, unas cien líneas se habrían copiado entre los dos diseños paralelos, y sin `engine/rules.hpp`, una corrección de una regla habría que repetirla tres veces.

Balance. D2 tiene la mejor relación entre rendimiento y complejidad en estos datos sobre Windows. D3 añade complejidad y no compensa con tiempo, lo cual no invalida a OpenMP, sino al rebalanceo en estos mundos: la comparación con su control estático lo muestra. D1 es el mejor mientras el mundo sea pequeño, y siempre es necesario como referencia.

10 Experimentos del ecosistema

10.1 Qué se esperaba y qué se encontró

> **El enunciado espera patrones cíclicos en los que zorros y conejos suben y bajan uno en relación con el otro. Esto es lo que muestran los datos:** las poblaciones sí suben y bajan de forma periódica, pero con el periodo corto de reproducción de cada especie, `GEN_PROC+1` generaciones, y no con el ciclo lento depredador-presa esperado. Con `GEN_PROC_FOXES=10` aparece una oscilación lenta débil, con los zorros detrás de los conejos, que no demuestra una interacción depredador-presa. En las 39 corridas de barrido no se vio ningún ciclo lento fuerte, y solo 2 tienen un periodo lento débil con el umbral usado. Esto vale a la sensibilidad indicada en 10.8: no prueba que no exista nada más lento.

Las reglas no se ajustaron para obtener ciclos: están fijadas por las salidas esperadas de la tarea, y la configuración base se eligió antes de los barridos y no se cambió después.

10.2 Método

- **Corridas.** Mundo 100x100 con `eco_seq` y `--stats-file`, 10 000 generaciones, semillas 1, 2 y 3, que fijan las posiciones iniciales.
- **Base.** 1500 conejos, 150 zorros y 500 rocas (5 %), `GEN_PROC_RABBITS=3`, `GEN_PROC_FOXES=20` y `GEN_FOOD_FOXES=10`, los valores de los ejemplos de la tarea. Se eligió una sola vez, tras explorar 10 combinaciones iniciales, en todas las cuales coexistieron las dos especies.
- **Barridos.** Un parámetro cada vez desde la base, con valores fijados de antemano: 13 configuraciones por 3 semillas, 39 corridas. Se añadieron 6 corridas largas de 20 000 generaciones, de la base y de `GEN_PROC_FOXES=10`, después de ver una señal débil en esta última.
- **Análisis** (`experiments/ecosystem_runs.py`). Se descarta el primer 20 % y la tendencia lineal. El periodo rápido es el primer pico de autocorrelación >= 0,3 de la serie cruda. Ese patrón se resta exactamente, restando a cada generación la media de su fase módulo el mínimo común múltiplo de los periodos rápidos (el *fold*: 84 en la base). En lo que queda se busca un periodo lento: **débil** si el pico de autocorrelación es >= 0,3, y **fuerte** si además llega a 0,6 con amplitud de al menos 10 % de la media.

Los datos de cada corrida están en `experiments/results/ecosystem`, y `ecosystem_findings.md` tiene el detalle.

10.3 Oscilación rápida

Los periodos de la serie cruda son iguales en las 3 semillas: `GEN_PROC_RABBITS+1` en conejos (3, 4 o 6) y `GEN_PROC_FOXES+1` en zorros (11, 21 o 41). Es una oscilación grande: en la base, entre las generaciones 2000 y 2100, los conejos van de 1238 a 2724 y los zorros hacen dientes de sierra de 310 a 681 (figura 11). El coeficiente de variación de la serie cruda es de 0,23 en conejos y 0,21 en zorros. La coincidencia con `GEN_PROC+1` es lo observado; el mecanismo no se verificó, ni se demostró que esta oscilación sea una interacción depredador-presa.

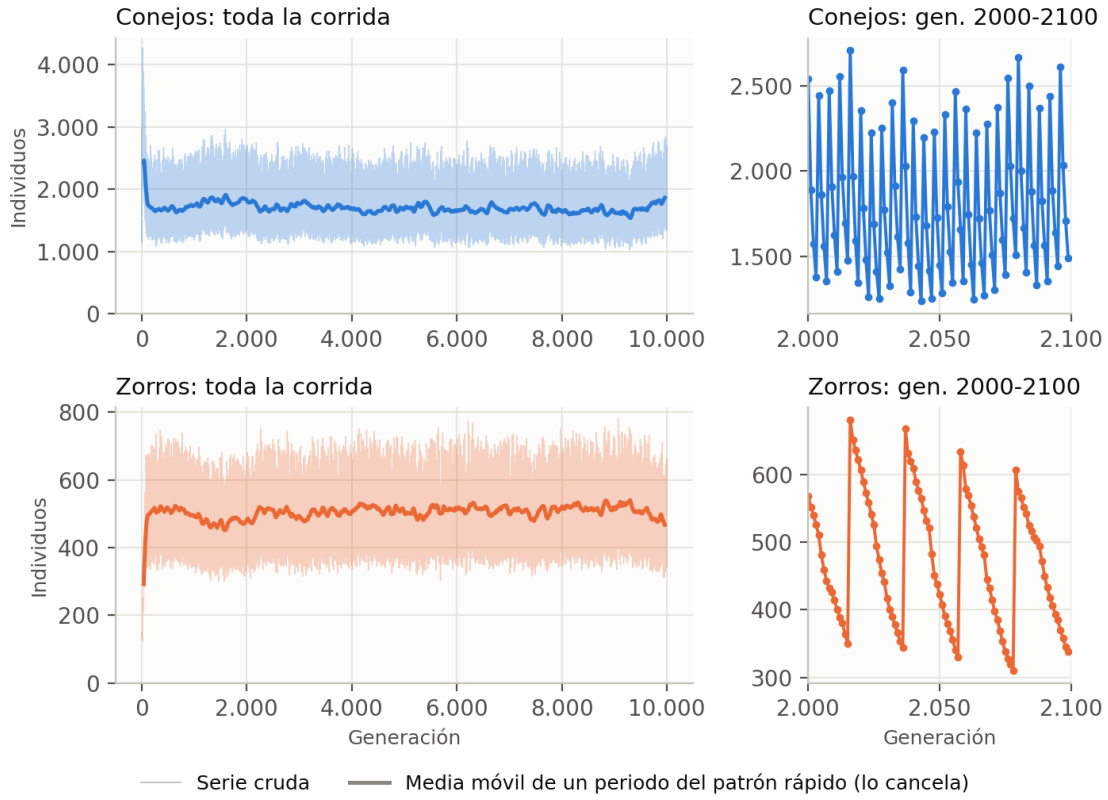


Figura 11. Configuración base, semilla 1. A la derecha se ve el detalle del patrón rápido en 100 generaciones.

10.4 Un solo sobreimpulso inicial

Al empezar, la población hace un único sobreimpulso amortiguado, no un ciclo que se repita (figura 12). En la base (semilla 1), los conejos suben de 1500 a un máximo de 4262 en la generación 20, mientras los zorros bajan a 126. Hacia la generación 110 los zorros han subido a unos 496 y los conejos han bajado a unos 1767 (medias móviles de 21 generaciones). Hay un rebote pequeño (1667 en la 210 y 1767 en la 310), y luego se estabilizan (1720 en la 1010).

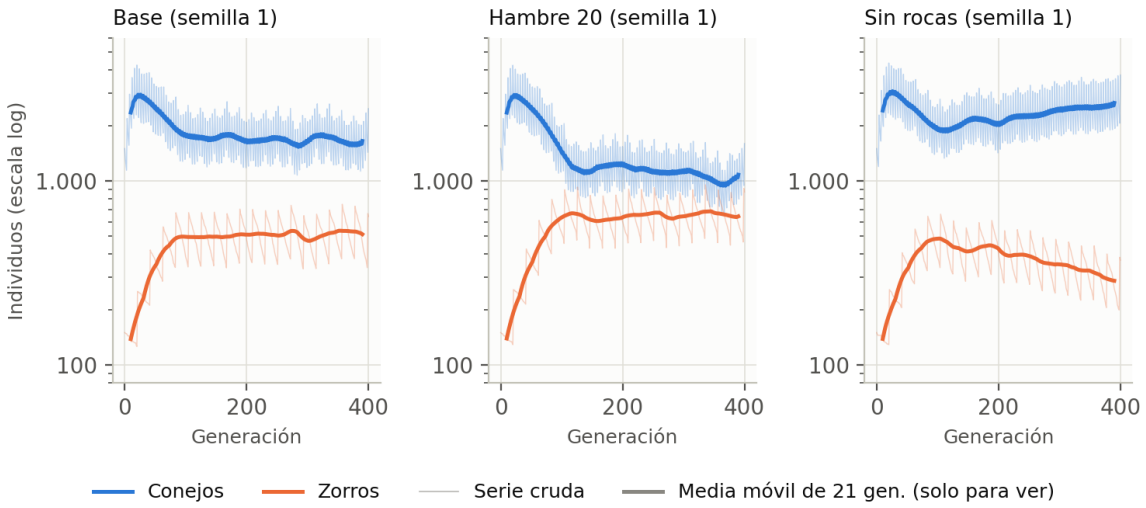


Figura 12. Primeras 400 generaciones de tres configuraciones, semilla 1.

10.5 Oscilación lenta débil con `GEN_PROC_FOXES=10`

Tabla 18. Periodos lentos con `GEN_PROC_FOXES=10`, cuyo *fold* es 44. Pares conejos / zorros. El desfase positivo significa que los zorros van detrás.

Corrida	Periodo conejos	Periodo zorros	Pico de autocorrelación	Amplitud (% de la media)	Grado	Desfase zorros
10 000, semilla 1	-	74	- / 0,35	- / 4,6	débil	22
10 000, semilla 2	77	76	0,37 / 0,44	8,9 / 5,8	débil	22
10 000, semilla 3	-	-	- / -	- / -	ninguno	-
20 000, semilla 1	78	77	0,40 / 0,45	8,5 / 5,5	débil	23
20 000, semilla 2	77	76	0,33 / 0,41	8,6 / 5,7	débil	22
20 000, semilla 3	79	76	0,32 / 0,40	8,7 / 5,5	débil	23

Con umbral 0,3 se detecta en 5 de las 6 corridas. La semilla 3 de 10 000 generaciones queda por debajo del umbral; con 0,2 da periodos de 79 y 75 y desfase de 22. En las 6, los zorros van 22 a 23 generaciones detrás de los conejos, con correlación cruzada de 0,92 a 0,94, y amplitud de 5 a 9 % de la media.

Esto no demuestra una interacción depredador-presa. Es débil, y los periodos hallados están cerca de múltiplos de los periodos rápidos 4 y 11 (76 = 19 × 4, 77 = 7 × 11), de modo que no se descarta un batido entre los dos temporizadores de reproducción. No se probó.

10.6 Efecto de los parámetros en la estabilidad

Tabla 19. Resultado de los barridos, con el rango de las 3 semillas. `slowcv` es la variación lenta: desviación entre media de la segunda mitad sin patrón rápido ni tendencia. Generación de extinción en cada semilla.

Configuración	Coexisten	Conejos (media)	Zorros (media)	`slowcv` conejos / zorros	Extinción: generación, semillas 1/2/3
Base: 3/20/10, 5 % de rocas	3 de 3	1684 a 1830	472 a 508	0,033 a 0,057 / 0,026 a 0,058	ninguna
`GEN_PROC_RABBITS=2`	3 de 3	1828 a 1836	677 a 682	0,035 a 0,039 / 0,023 a 0,024	ninguna
`GEN_PROC_RABBITS=5`	3 de 3	2572 a 2655	27 a 52	0,007 a 0,011 / 0,140 a 0,204	ninguna
`GEN_PROC_RABBITS=8`	0 de 3	1795 a 1870	0 a 4	-	zorros: 9171 / 2915 / 1974
`GEN_PROC_FOXES=5`	0 de 3	0	126 a 152	-	conejos: 79 / 493 / 71
`GEN_PROC_FOXES=10`	3 de 3	935 a 950	653 a 655	0,058 a 0,070 / 0,031 a 0,040	ninguna
`GEN_PROC_FOXES=40`	3 de 3	3319 a 3489	84 a 110	0,018 a 0,025 / 0,079 a 0,163	ninguna
`GEN_FOOD_FOXES=3`	0 de 3	3603 a 3605	0	-	zorros: 112 / 101 / 123
`GEN_FOOD_FOXES=6`	3 de 3	2427 a 2660	262 a 324	0,027 a 0,036 / 0,055 a 0,089	ninguna
`GEN_FOOD_FOXES=20`	3 de 3	1151 a 1290	612 a 644	0,049 a 0,077 / 0,023 a 0,035	ninguna
Rocas 0 %	3 de 3	3660 a 3678	10 a 15	0,004 a 0,005 / 0,441 a 0,530	ninguna
Rocas 15 %	3 de 3	1474 a 1487	501 a 503	0,019 a 0,025 / 0,014 a 0,019	ninguna
Rocas 30 %	3 de 3	1535 a 1551	359 a 369	0,023 a 0,026 / 0,024 a 0,026	ninguna

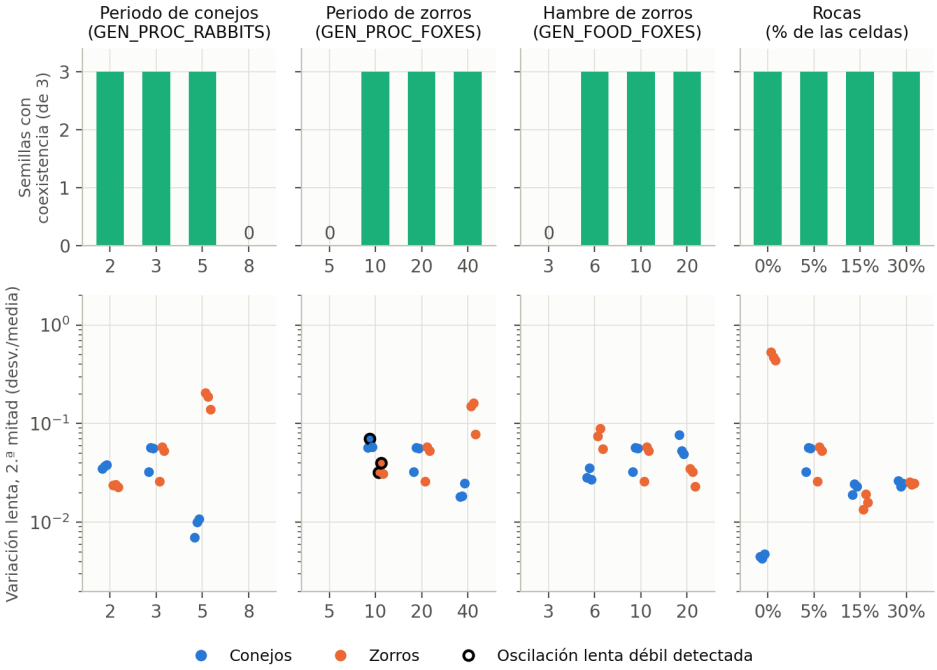


Figura 13. Coexistencia y variación lenta por parámetro. Con círculo negro, las corridas con oscilación lenta débil detectada.

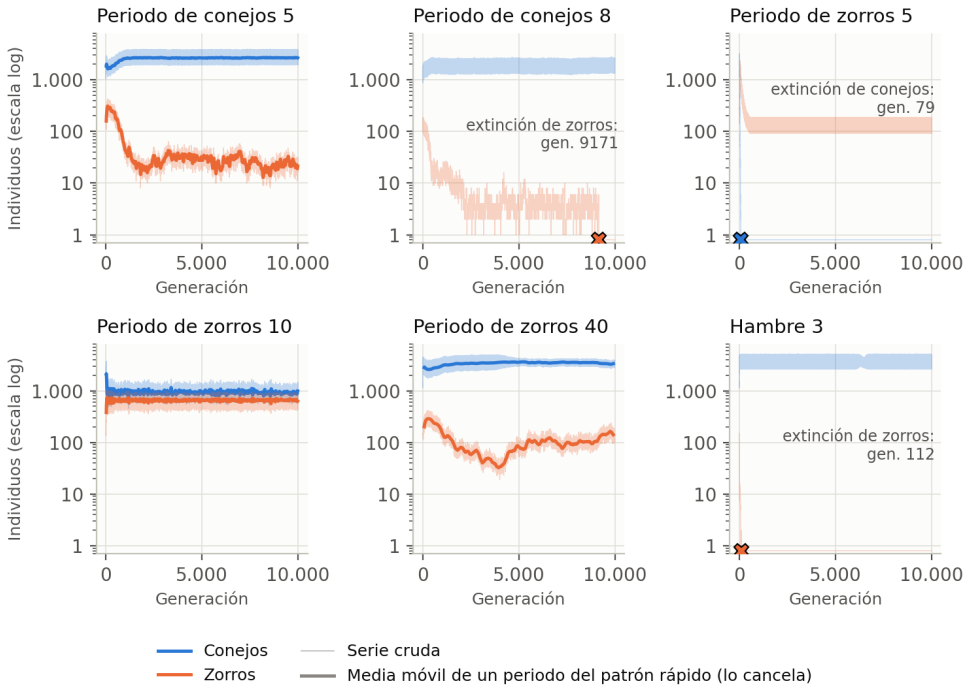


Figura 14. Series de población al variar los parámetros.

Estas son **asociaciones; no se probó causalidad**:

- `GEN_PROC_RABBITS`**.** Los zorros bajan de 677 a 682 con 2 a 27 a 52 con 5. Con 8 se extinguen, en las generaciones 9171, 2915 y 1974 según la semilla.
- `GEN_PROC_FOXES`**.** Con 5 se extinguen los conejos, en las generaciones 79, 493 y 71, y sobreviven los zorros. Una prueba aparte, sin conejos, también deja zorros con ese valor (72 en la generación 2999), mientras que con 10 se extinguen en la generación 10. La causa no se probó. Con 10 hay más zorros (653 a 655) y menos conejos (935 a 950).
- `GEN_FOOD_FOXES`**.** Con 3 los zorros se extinguen en las generaciones 112, 101 y 123. Los zorros suben de 262 a 324 con 6 a 612 a 644 con 20, y los conejos bajan.

- **Derivas largas.** "Estable" no significa "constante". Por ejemplo, con `GEN_PROC_RABBITS=5` (semilla 1) los zorros pasan de unos 302 en la generación 150 a unos 23 en la 1510.

10.7 Efecto de las rocas

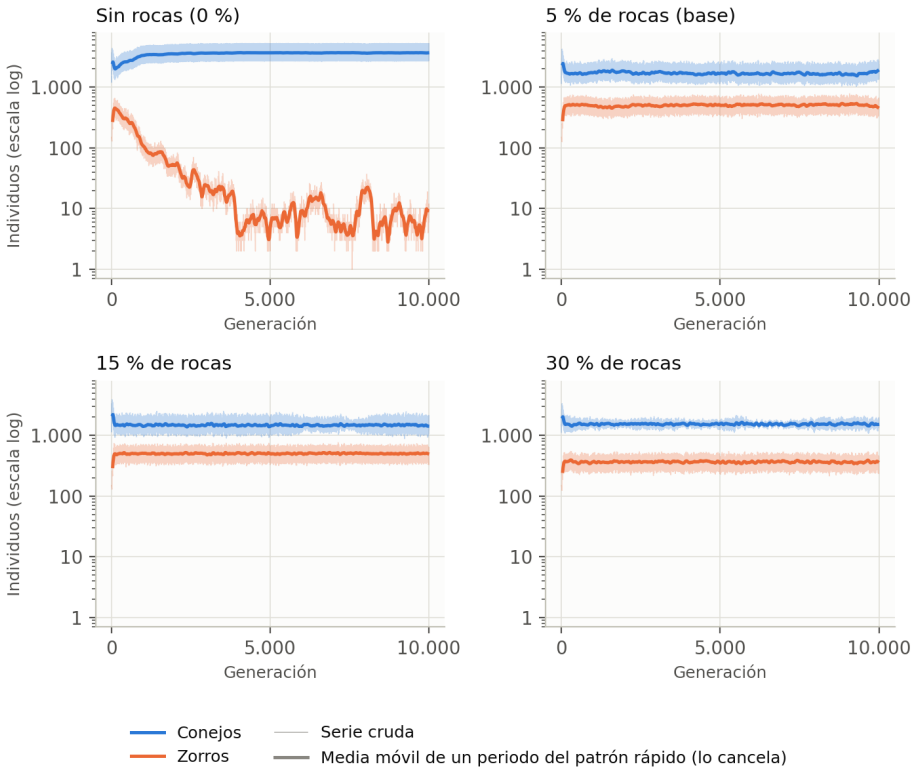


Figura 15. Series de población con 0, 5, 15 y 30 % de rocas.

- **Sin rocas**, los zorros suben a unos 485 hacia la generación 105 y luego declinan durante miles de generaciones (109 en la 1010 y 21 en la 3010). Quedan con una media de 10 a 15, con altibajos irregulares: no se extinguen en 10 000 generaciones, pero quedan cerca de 0.
- **Con rocas**, los zorros se sostienen: 472 a 508 con 5 %, 501 a 503 con 15 % y 359 a 369 con 30 %. La variación lenta de los zorros es la más baja con 15 % (0,014 a 0,019, frente a 0,026 a 0,058 con 5 %).
- **Son asociaciones.** El enunciado describe las rocas como refugio, pero que actúen como refugio de los conejos o que rompan la sincronía espacial es una hipótesis no probada: no se midió la distribución espacial.

10.8 Qué detecta el análisis, y qué no

Para saber qué se habría visto, se inyectaron ciclos conocidos en series reales de 4 configuraciones y 3 semillas: se multiplicó la serie por (1 + a sen(2π g / P)) y se contó cuántas veces el análisis encontró un periodo a menos del 15 % de P.

Tabla 20. Detección de ciclos inyectados (`experiments/results/ecosystem_injection.csv`).

Periodo inyectado	Amplitud 2 %	Amplitud 5 %	Amplitud 10 %	Amplitud 20 %
40	6 de 12	9 de 12	11 de 12	12 de 12
60	6 de 12	9 de 12	11 de 12	12 de 12
84	3 de 12	3 de 12	3 de 12	8 de 12
100	6 de 12	10 de 12	11 de 12	12 de 12
150	6 de 12	10 de 12	11 de 12	12 de 12
300	6 de 12	10 de 12	11 de 12	12 de 12
600	6 de 12	10 de 12	12 de 12	12 de 12
1000	6 de 12	10 de 12	12 de 12	12 de 12
2000	5 de 12	10 de 12	11 de 12	12 de 12

- **Sensibilidad.** Con amplitud de al menos 10 % de la media se detectan casi todos entre 40 y 2000 generaciones, salvo el periodo 84. Un ciclo de 5 % o menos puede pasar inadvertido.
- **Hay un punto ciego en el fold.** Un ciclo cuyo periodo coincide con el *fold* (84 en tres de las cuatro configuraciones) se resta junto con el patrón: hasta 10 % de amplitud solo se detectó donde el *fold* era otro, y con 20 %, en 5 de las 9 series con *fold* 84. En particular, en la serie cruda de la base larga hay un pico de autocorrelación en 84 (0,96 a 0,97), pero 84 es el mínimo común múltiplo de 4 y 21, así que ese pico es el que produce el patrón rápido. Cualquier señal real cerca de 84-85 no se puede distinguir de esa banda. La rejilla de inyección tampoco tiene otros periodos entre 60 y 100.
- **Umbral.** De las 39 corridas de barrido, 0 tienen periodo lento con umbral 0,5, 2 con 0,3, 7 con 0,2 y 22 con 0,1. Con 0,1 los periodos van de unos 76 a 2600 generaciones, con desfases de signo cambiante, lo que parece ruido.
- **Falsos positivos.** En 150 series de ruido rojo (AR(1), phi 0,99) se halló un ciclo falso en 6 con 6000 generaciones y en 0 con 10 000.
- **Sin probar.** Otros tamaños de mundo, otras densidades iniciales, corridas de más de 20 000 generaciones y periodos menores que 40. Tampoco se probaron las causas de la oscilación rápida, de la débil, de las derivas ni del efecto de las rocas.

11 Interfaz visual

La interfaz es una página web local en tres columnas: parámetros a la izquierda, grilla al centro con los controles de tiempo debajo, y estadísticas a la derecha. Funciona sin internet, con Python y su biblioteca estándar en el servidor y HTML, CSS y JavaScript sin dependencias externas en el navegador.

El servidor (`web/server.py`) lanza el motor como proceso aparte con `--frames`. Cada cuadro tiene una cabecera de 16 bytes y 1 byte por celda. El motor se bloquea cuando nadie lee, así que una página en pausa no acumula memoria, y una corrida larga no se genera por adelantado. Por omisión usa `eco_seq`. Las mediciones de 8 no pasan por la interfaz.

Tabla 21. Los cuatro requisitos visuales del enunciado.

Requisito	Cómo se cumple
Grilla con todas las entidades	Canvas central, con color distinto para conejo, zorro y roca, y con leyenda. Desde 8 píxeles por celda usa también formas. Por debajo, solo el color los distingue.
Estadísticas de población en el tiempo	Contadores de conejos, zorros y rocas, generación actual y gráfica de población. La gráfica dibuja una banda de mínimo y máximo con la media, para que el patrón rápido no la vuelva una mancha.
Parámetros ajustables	Poblaciones iniciales, tamaño y semilla en el mundo aleatorio; `GEN_PROC_RABBITS`, `GEN_PROC_FOXES`, `GEN_FOOD_FOXES` y `N_GEN` también en los casos de prueba. Cada uno lleva su nombre en lenguaje claro y su nombre técnico.
Controles de tiempo	Pausa y reanudar, también con la barra espaciadora; "Más lento" y "Más rápido", de 1 a 500 generaciones por segundo; y barra de progreso.

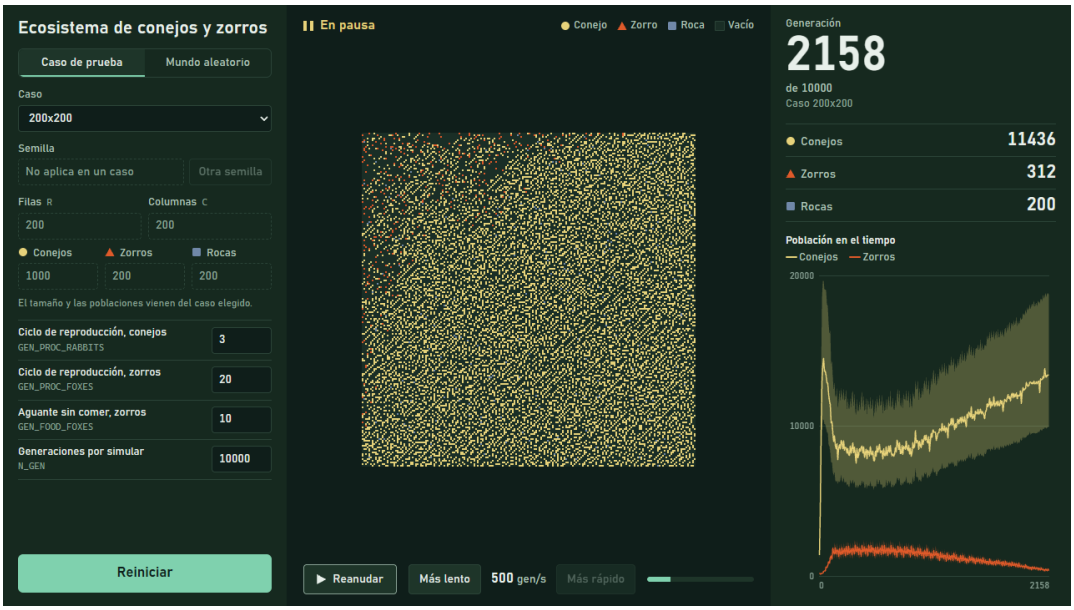


Figura 16. Caso 200x200 en pausa a 500 generaciones por segundo. Abajo, los controles de tiempo. A la derecha, los contadores y la gráfica de población.

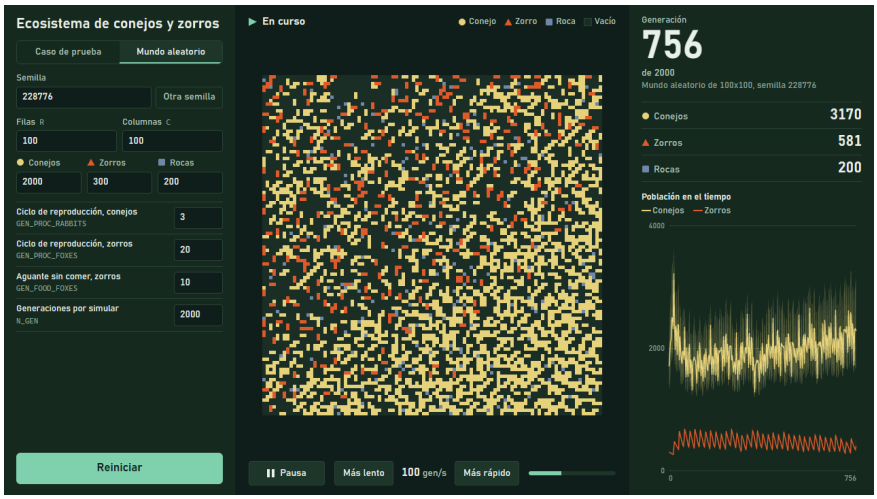


Figura 17. Mundo aleatorio con semilla visible, para poder repetir un experimento.

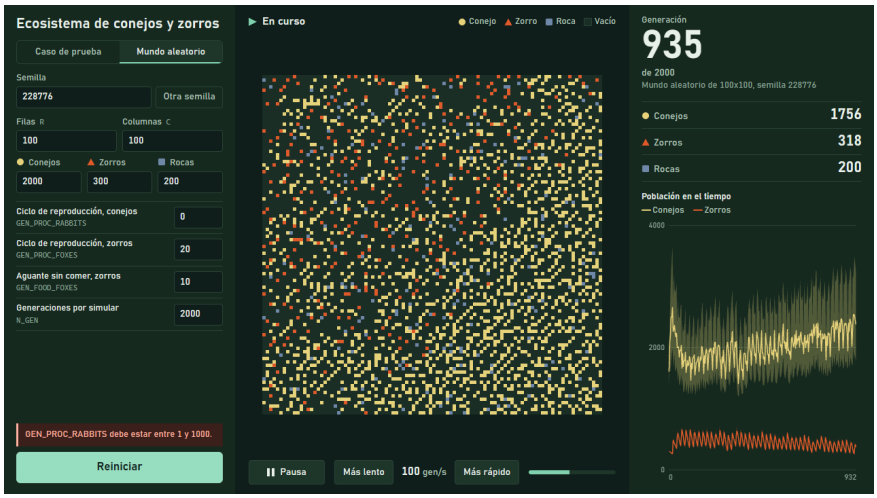


Figura 18. Un parámetro inválido muestra un mensaje claro y la corrida en curso continúa.

La interfaz se revisó en Chromium: un caso de 5x5 hasta el final, con conteos iguales a la salida esperada; 200x200 a 500 generaciones por segundo hasta la generación 10 000, sin errores en la página; pausa, velocidades, reinicio, parámetros inválidos y motor sin compilar. El límite es de 500 filas y 500 columnas. El código de la página no tiene pruebas automáticas; sí las tiene el servidor (`tests/test_server.py`).

12 Cómo ejecutar el programa

Requisitos: g++ con OpenMP y Pthreads (se usó WinLibs GCC 16.2.0), CMake y Ninja, y Python 3.12. El script de compilación fija la ruta de WinLibs de este equipo, de modo que en otro hay que ajustarla.

```
# Compilar en Windows (deja eco_seq, eco_pth y eco_omp en build/)
powershell scripts/build.ps1

# Compilar en Linux
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release -DECO_STATIC=OFF && cmake --build build

# Ejecutar cada diseño: entrada por stdin, resultado por stdout, tiempos por stderr
build/eco_seq.exe < ecosystem_examples/input100x100
build/eco_pth.exe --threads 8 < ecosystem_examples/input100x100
build/eco_omp.exe --threads 8 < ecosystem_examples/input100x100
build/eco_omp.exe --threads 8 --schedule static < ecosystem_examples/input100x100

# Interfaz: abrir http://127.0.0.1:8000 en el navegador
python web/server.py --port 8000

# Pruebas
python -m unittest discover -s tests -t .

# Repetir las mediciones y los experimentos
python experiments/measure.py run
python experiments/ecosystem_runs.py run

# Regenerar este informe
python docs/report/build_report.py
```

La línea de tiempos en stderr tiene esta forma: `time wall=... cpu=... threads=... imbalance=...`. Otras opciones del motor: `--dump-all`, que escribe todas las generaciones en el formato `allgen`; `--frames`, que emite cuadros para la interfaz; y `--stats-file RUTA`, que guarda la población por generación.

13 Conclusión

La reflexión del curso dice que un diseño paralelo no siempre necesita más hilos para ser más rápido. Estos datos lo confirman de forma concreta. Con los mismos 16 hilos, Pthreads tarda 0,260 s en 20x20, frente a 0,004 s del secuencial, y en 1000x1000 acelera ×4,27. Lo que decide no es el número de hilos, sino cuánto trabajo hay entre dos puntos de sincronización.

Tres conclusiones van más allá de lo que se esperaba al empezar:

1. **La complejidad no compró tiempo.** El reparto adaptativo funciona como mecanismo, porque baja el desequilibrio, pero en estos casos el desequilibrio no era el problema.
2. **La plataforma importa tanto como el diseño.** La misma fuente invierte la comparación entre OpenMP y Pthreads al cambiar de entorno, de modo que una elección de modelo basada en una sola plataforma no se puede generalizar.
3. **Separar decisión y entrega fue lo más valioso.** Decidir sobre un estado de solo lectura y entregar de forma conmutativa dio el mismo resultado en todos los casos y números de hilos, con una sincronización simple y sin candados.

Tabla 22. Recomendación según el tamaño del mundo, con los datos de Windows de tabla 12.

Mundo	Diseño recomendado	Por qué
Pequeño (hasta unos 20x20)	D1, secuencial	Cualquier diseño paralelo es más lento: 0,004 s frente a 0,022 s con Pthreads y 2 hilos.
Mediano (100x100)	D2 con 4 hilos	Único con ganancia: ×1,66. Con 2 a 4 hilos basta, y con 16 es peor que el secuencial. OpenMP no pasa de ×0,96.
Mediano-grande (200x200)	D2 con 8 hilos	×2,70. Con 16 hilos baja a ×1,79.
Grande (1000x1000)	D2 o D3 con 8 hilos	×4,14 y ×3,94. Pasar a 16 hilos casi no mejora el tiempo y sube la CPU de 14,875 s a 25,625 s.

Estas recomendaciones valen para este equipo y este sistema. En el contenedor Linux, en 200x200, la recomendación sería OpenMP (×4,23 con 8 hilos, frente a ×0,95 de Pthreads). Por eso, antes de elegir, conviene medir en la plataforma final. No se recomienda el rebalanceo por su tiempo: su control estático rinde igual en estos casos, con menos código.

En cuanto al ecosistema, la simulación muestra cómo los parámetros cambian la estabilidad, desde la coexistencia hasta la extinción de una especie, y sus poblaciones oscilan. Pero no muestra, a la sensibilidad indicada, el ciclo lento y fuerte entre depredador y presa que esperaba el enunciado, y este informe lo deja dicho en lugar de forzarlo.

14 Apéndice: tablas completas de tiempos

Cada celda tiene la mediana del tiempo total en segundos, el IQR entre corchetes, la aceleración respecto a `eco_seq` (×) y la eficiencia (e). La última columna es el sobrecoste con 1 hilo frente a `eco_seq`. El tiempo de CPU de cada medición está en `experiments/results/performance.csv`, y aparece en figura 7.

14.1 Windows 10 nativo

Tabla 23. 20x20: `eco_seq` 0,004 s [0,000], 1000 generaciones. El CPU medido aquí no es fiable, por su resolución.

Diseño	1 hilo	2 hilos	4 hilos	8 hilos	16 hilos	Extra a 1 hilo
Pthreads (D2)	0,005 [0,000] ×0,77, e 0,77	0,022 [0,001] ×0,18, e 0,09	0,031 [0,001] ×0,13, e 0,03	0,095 [0,001] ×0,04, e 0,01	0,260 [0,002] ×0,01, e 0,00	+30,3 %
OpenMP adaptativo (D3)	0,005 [0,000] ×0,75, e 0,75	0,051 [0,001] ×0,08, e 0,04	0,098 [0,001] ×0,04, e 0,01	0,197 [0,001] ×0,02, e 0,00	0,382 [0,002] ×0,01, e 0,00	+32,5 %
OpenMP estático (control)	0,005 [0,000] ×0,75, e 0,75	0,051 [0,000] ×0,08, e 0,04	0,097 [0,001] ×0,04, e 0,01	0,195 [0,001] ×0,02, e 0,00	0,379 [0,002] ×0,01, e 0,00	+33,2 %

Tabla 24. 100x100: `eco_seq` 1,043 s [0,011], 10000 generaciones.

Diseño	1 hilo	2 hilos	4 hilos	8 hilos	16 hilos	Extra a 1 hilo
Pthreads (D2)	1,211 [0,006] ×0,86, e 0,86	0,861 [0,006] ×1,21, e 0,61	0,630 [0,031] ×1,66, e 0,41	0,916 [0,060] ×1,14, e 0,14	2,260 [0,003] ×0,46, e 0,03	+16,1 %
OpenMP adaptativo (D3)	1,229 [0,016] ×0,85, e 0,85	1,106 [0,009] ×0,94, e 0,47	1,100 [0,012] ×0,95, e 0,24	1,756 [0,042] ×0,59, e 0,07	3,703 [0,007] ×0,28, e 0,02	+17,9 %
OpenMP estático (control)	1,223 [0,006] ×0,85, e 0,85	1,111 [0,018] ×0,94, e 0,47	1,090 [0,005] ×0,96, e 0,24	1,789 [0,044] ×0,58, e 0,07	3,709 [0,068] ×0,28, e 0,02	+17,3 %

Tabla 25. 100x100 `unbal01`: `eco_seq` 0,746 s [0,008], 10000 generaciones.

Diseño	1 hilo	2 hilos	4 hilos	8 hilos	16 hilos	Extra a 1 hilo
Pthreads (D2)	0,897 [0,005] ×0,83, e 0,83	0,698 [0,002] ×1,07, e 0,53	0,559 [0,007] ×1,33, e 0,33	0,873 [0,004] ×0,85, e 0,11	2,339 [0,061] ×0,32, e 0,02	+20,3 %
OpenMP adaptativo (D3)	0,904 [0,010] ×0,83, e 0,83	0,946 [0,013] ×0,79, e 0,39	1,013 [0,007] ×0,74, e 0,18	1,806 [0,034] ×0,41, e 0,05	3,755 [0,032] ×0,20, e 0,01	+21,2 %
OpenMP estático (control)	0,901 [0,007] ×0,83, e 0,83	0,949 [0,011] ×0,79, e 0,39	1,006 [0,006] ×0,74, e 0,19	1,772 [0,012] ×0,42, e 0,05	3,715 [0,042] ×0,20, e 0,01	+20,8 %

Tabla 26. 100x100 `unbal02`: `eco_seq` 1,019 s [0,012], 10000 generaciones.

Diseño	1 hilo	2 hilos	4 hilos	8 hilos	16 hilos	Extra a 1 hilo
Pthreads (D2)	1,203 [0,017] ×0,85, e 0,85	0,863 [0,003] ×1,18, e 0,59	0,652 [0,024] ×1,56, e 0,39	0,974 [0,076] ×1,05, e 0,13	2,321 [0,050] ×0,44, e 0,03	+18,0 %
OpenMP adaptativo (D3)	1,213 [0,017] ×0,84, e 0,84	1,109 [0,006] ×0,92, e 0,46	1,101 [0,007] ×0,93, e 0,23	1,768 [0,036] ×0,58, e 0,07	3,736 [0,105] ×0,27, e 0,02	+18,9 %
OpenMP estático (control)	1,214 [0,007] ×0,84, e 0,84	1,116 [0,012] ×0,91, e 0,46	1,088 [0,009] ×0,94, e 0,23	1,739 [0,028] ×0,59, e 0,07	3,705 [0,041] ×0,28, e 0,02	+19,0 %

Tabla 27. 200x200: `eco_seq` 3,999 s [0,015], 10000 generaciones.

Diseño	1 hilo	2 hilos	4 hilos	8 hilos	16 hilos	Extra a 1 hilo
Pthreads (D2)	4,673 [0,025] ×0,86, e 0,86	2,716 [0,013] ×1,47, e 0,74	1,663 [0,097] ×2,41, e 0,60	1,481 [0,093] ×2,70, e 0,34	2,230 [0,032] ×1,79, e 0,11	+16,8 %
OpenMP adaptativo (D3)	4,682 [0,025] ×0,85, e 0,85	2,942 [0,008] ×1,36, e 0,68	2,071 [0,048] ×1,93, e 0,48	2,110 [0,070] ×1,90, e 0,24	3,393 [0,122] ×1,18, e 0,07	+17,1 %
OpenMP estático (control)	4,686 [0,016] ×0,85, e 0,85	2,981 [0,045] ×1,34, e 0,67	2,061 [0,036] ×1,94, e 0,49	2,239 [0,177] ×1,79, e 0,22	3,384 [0,199] ×1,18, e 0,07	+17,2 %

Tabla 28. 1000x1000: eco_seq 8,688 s [0,155], 1000 generaciones.

Diseño	1 hilo	2 hilos	4 hilos	8 hilos	16 hilos	Extra a 1 hilo
Pthreads (D2)	10,227 [0,014] ×0,85, e 0,85	5,469 [0,080] ×1,59, e 0,79	3,184 [0,167] ×2,73, e 0,68	2,098 [0,090] ×4,14, e 0,52	2,032 [0,043] ×4,27, e 0,27	+17,7 %
OpenMP adaptativo (D3)	10,312 [0,142] ×0,84, e 0,84	5,402 [0,026] ×1,61, e 0,80	3,102 [0,124] ×2,80, e 0,70	2,203 [0,104] ×3,94, e 0,49	2,097 [0,151] ×4,14, e 0,26	+18,7 %
OpenMP estático (control)	10,252 [0,076] ×0,85, e 0,85	5,442 [0,148] ×1,60, e 0,80	3,158 [0,040] ×2,75, e 0,69	2,340 [0,158] ×3,71, e 0,46	2,123 [0,184] ×4,09, e 0,26	+18,0 %

14.2 Linux en un contenedor sobre WSL2

Tabla 29. 200x200: eco_seq 3,958 s [0,037], kernel 6.18.54.1-microsoft-standard-WSL2.

Diseño	1 hilo	2 hilos	4 hilos	8 hilos	16 hilos	Extra a 1 hilo
Pthreads (D2)	4,957 [0,024] ×0,80, e 0,80	3,373 [0,024] ×1,17, e 0,59	3,351 [0,103] ×1,18, e 0,30	4,165 [0,259] ×0,95, e 0,12	6,902 [0,220] ×0,57, e 0,04	+25,2 %
OpenMP adaptativo (D3)	4,860 [0,022] ×0,81, e 0,81	2,722 [0,084] ×1,45, e 0,73	1,619 [0,026] ×2,45, e 0,61	0,936 [0,006] ×4,23, e 0,53	0,587 [0,214] ×6,74, e 0,42	+22,8 %
OpenMP estático (control)	4,876 [0,034] ×0,81, e 0,81	2,761 [0,041] ×1,43, e 0,72	1,649 [0,027] ×2,40, e 0,60	0,939 [0,129] ×4,22, e 0,53	0,841 [0,430] ×4,71, e 0,29	+23,2 %

Tabla 30. 1000x1000: eco_seq 9,740 s [0,029].

Diseño	1 hilo	2 hilos	4 hilos	8 hilos	16 hilos	Extra a 1 hilo
Pthreads (D2)	11,849 [0,184] ×0,82, e 0,82	5,928 [0,162] ×1,64, e 0,82	3,664 [0,096] ×2,66, e 0,66	2,580 [0,277] ×3,78, e 0,47	2,483 [0,089] ×3,92, e 0,25	+21,7 %
OpenMP adaptativo (D3)	11,829 [0,077] ×0,82, e 0,82	6,163 [0,144] ×1,58, e 0,79	3,794 [0,130] ×2,57, e 0,64	2,508 [0,058] ×3,88, e 0,49	2,384 [0,459] ×4,09, e 0,26	+21,4 %
OpenMP estático (control)	11,845 [0,129] ×0,82, e 0,82	6,227 [0,122] ×1,56, e 0,78	3,866 [0,112] ×2,52, e 0,63	2,612 [0,059] ×3,73, e 0,47	2,426 [0,233] ×4,01, e 0,25	+21,6 %